

Danmarks
Tekniske
Universitet



Bachelor Project

Iterative Solvers for Large Linear Systems of Equations

JACOB BRÜNNICH KNUDSEN

Student ID: s224372

Department of Applied Mathematics and Computer Science

Supervisor: Martin Skovgaard Andersen

Technical University of Denmark

January 6, 2025

Abstract

In this project we investigate two different preconditioners used for solving large-scale symmetric positive semidefinite (psd) linear systems via the Conjugate Gradient (CG) method. Both preconditioners utilize the Nyström approximation to construct a low rank psd approximation of the system matrix. The first preconditioner, which we will refer to as the randomized Nyström preconditioner, uses the randomized Nyström approximation to construct the preconditioner. The second preconditioner relies on the column Nyström approximation with a probabilistic pivot rule to select the columns, hence its name Random Pivoted Cholesky (RPCholesky) preconditioner. We provide both theoretical underpinnings and practical implementations of these methods, focusing on iteration counts and total CPU time for convergence. Numerical experiments on large Gaussian kernel matrices (up to size $n = 1000$) indicate that while both preconditioners improve convergence rates relative to the unpreconditioned system, RPCholesky generally performs faster in terms of total CPU time. In particular, as the rank of the approximation increases, the randomized Nyström method exhibits a large overhead cost for constructing the preconditioner, which ultimately outweighs its gains in iteration count reduction. By contrast, RPCholesky maintains low construction overhead and scales better to higher ranks, making it the more practical choice for large-scale applications involving Gaussian kernel system matrices.

Contents

1	Introduction	1
2	Theory	2
2.1	Cholesky Decomposition	2
2.2	Partial Cholesky Decomposition	3
2.3	Important Properties of Symmetric psd Matrices	3
2.4	Conjugate Gradient Method	5
2.5	Nyström Approximation	6
2.6	Cayley-Hamilton Theorem	7
2.7	Krylov Subspaces	8
2.8	Convergence Analysis of the Conjugate Gradient Method	9
2.8.1	Condition number	9
2.8.2	Spectrum and right hand side	10
2.9	Preconditioning	12
2.10	Randomized Nyström Preconditioning	14
2.11	Pivoted partial cholesky algorithm	16
2.12	Boxplots	18
3	Experiments and Results	19
3.1	Experimental Setup	19
3.1.1	Problem Definition	19
3.1.2	Test Matrices	19
3.1.3	Experimental Parameters	20
3.2	Implementation Details	21
3.3	Results	21
4	Discussion	26
5	Conclusion	27
	List of Figures	I
	List of Tables	II
	References	III
6	Appendix	V
7	Code appendix	VI
7.1	Code Listing	VI

1 Introduction

Systems of linear equations

$$Ax = b,$$

where $A \in \mathbb{R}^{n \times n}$ is symmetric and positive definite, arise in a wide range of important scientific and engineering applications. Notable examples include partial differential equations, structural analysis and circuit analysis.[1] These matrices often grow to very large dimensions, making direct methods (e.g., Cholesky factorization) computationally expensive, and thus iterative methods become essential for practical solving. One of these iterative methods is the Conjugate Gradient (CG) algorithm, which exploits the symmetry and positive definiteness of A . CG iterates toward the solution by constructing and updating search directions that are A -orthogonal (conjugate) with respect to A , often converging in fewer than n steps (where n is the size of the matrix). The use of preconditioners aims to accelerate CG by transforming $Ax = b$ into an equivalent system

$$M^{-1}Ax = M^{-1}b,$$

where M is a carefully chosen matrix (the preconditioner) that is cheaper to invert than A . Ideally, this transformation brings the eigenvalues of $M^{-1}A$ closer together and thereby reduces the number of CG iterations required for convergence. In this thesis, we compare two different preconditioners both of which utilize randomness to make a low rank approximation of A . The first preconditioner is the Randomized Nyström preconditioner which is based on the Nyström method, that uses random sampling via a Gaussian test matrix, to identify a subspace of dimension ℓ (the sketch size). From this subspace, one obtains a rank- ℓ approximation

$$\hat{A}_{\text{nys}} = (A\Omega)(\Omega^T A\Omega)^\dagger (A\Omega)^T,$$

where $\Omega \in \mathbb{R}^{n \times \ell}$ is a random Gaussian matrix and $\dagger$ denotes the Moore–Penrose pseudoinverse. The method guarantees a positive semidefinite approximate factor for A , which can then serve as a preconditioner in CG. The second preconditioner is the Pivoted Partial Cholesky Decomposition, specifically the Random Pivoted Cholesky (RPCholesky) algorithm, which also uses the Nyström method but differs in the way it extracts information from A . In Particular, it adaptively selects columns of A as pivots according to a sampling distribution proportional to the diagonal entries of the current residual. In each step, the selected column (and corresponding row) is used to update a low-rank Cholesky factor

$$\hat{A} = FF^T,$$

where $F \in \mathbb{R}^{n \times \ell}$. The partial factor is easily transformed into an eigenvalue decomposition suitable for preconditioning. Our goal is to assess the effectiveness of these two approaches by looking at both:

1. iteration counts required to reduce the relative residual $\frac{\|b - Ax_k\|}{\|b\|}$ below a specified tolerance, and

2. the overall computational time, including the time to construct the preconditioner.

In order to compare the two preconditioners we run multiple experiments where we solve two different matrix systems using the preconditioners. The system matrices are Gaussian kernels constructed from real-world-inspired data sets: one that forms a smiley in $\mathbb{R}^2$, and one that forms a spiral. We run each method multiple times with varying ranks ℓ to capture how changes in the approximation quality affect convergence speed and preconditioning overhead.

The results show that both methods succeed in lowering the iteration count of the CG method compared to the unpreconditioned baseline. However, as the approximate rank ℓ grows, the Randomized Nyström method exhibits a large increase in CPU time—mainly due to the cost of computing the random sketch and performing the subsequent decompositions—whereas the RPCholesky approach remains really fast. Consequently, the RPCholesky preconditioner outperforms the Nyström preconditioner in terms of total computation time for the problems considered here.

2 Theory

2.1 Cholesky Decomposition

The Cholesky decomposition or Cholesky factorization is a matrix decomposition particularly suited for symmetric positive definite matrices which will prove very useful for this project later on. It can be viewed as a special case of LU decomposition for symmetric positive definite matrices. Recall that the LU decomposition factors a square matrix A as a product of a lower triangular matrix L and an upper triangular matrix U . But if A is symmetric positive definite then we can get the decomposition $A = LL^T$ utilizing the symmetry and positive definiteness of A .

The computation of the elements of L is straightforward. The diagonal elements of L is calculated using the formula:

$$l_{ii} = (\pm) \sqrt{a_{ii} - \sum_{k=1}^{i-1} l_{ik}^2} \quad (1)$$

and the off-diagonal elements l_{ij} where $i > j$ are computed as:

$$l_{ij} = \frac{1}{l_{jj}} \left(a_{ij} - \sum_{k=1}^{j-1} l_{ik} l_{jk} \right).$$

where the remaining elements l_{ij} , $i < j$ are all set to zero, making L lower triangular. It's important to note that the choice of the sign in equation (1) for the diagonal elements is arbitrary and does not affect the validity of the factorization, as long as we maintain consistency throughout the process. To preserve the correctness of the factorization when

changing a diagonal element's sign, we must also adjust the signs of all elements in the same column below that diagonal element.[2][3]

If you have the Cholesky decomposition of a system matrix matrix $LL^T = A$ where A is the coefficient matrix the system $Ax = b$ then by substitution we get $LL^Tx = b$. From this we see that x is easily computed in two steps:

1. Forward substitution: Solve for y

$$Ly = b$$

2. Backward substitution: Solve for x

$$L^Tx = y$$

2.2 Partial Cholesky Decomposition

While the full Cholesky decomposition described above provides an exact factorization, we will need a partial decomposition for applications later on. The partial Cholesky decomposition produces a low-rank approximation of a symmetric positive semidefinite matrix $A \in \mathbb{R}^{n \times n}$ by computing only the first k columns of the Cholesky factor, where $k < n$ [4]. This results in a factor $L_k \in \mathbb{R}^{n \times k}$ such that:

$$A \approx L_k L_k^T$$

The partial decomposition can be calculated using the same formulas as for the full Cholesky decomposition:

$$l_{ii} = (\pm) \sqrt{a_{ii} - \sum_{k=1}^{i-1} l_{ik}^2}$$

$$l_{ij} = \frac{1}{l_{jj}} \left(a_{ij} - \sum_{k=1}^{j-1} l_{ik} l_{jk} \right).$$

with the key difference that only the k first columns are calculated. Clearly k determines the accuracy of the decomposition here, with higher values of k yielding closer approximation of A until $k = n$ for which it becomes the full Cholesky decomposition.

2.3 Important Properties of Symmetric psd Matrices

We will now cover some important properties of symmetric positive semi-definite (psd) matrices which will be used extensively in this paper. Let us begin with the definition: A symmetric matrix A (that is, $A = A^T$ or equivalently, for all entries $a_{ij} = a_{ji}$) is positive semi-definite if for all vectors $x \in \mathbb{R}^n$, it holds that

$$x^T A x \geq 0. \tag{2}$$

This definition immediately implies that A must be square, as the matrix product $x^T A x$ is only defined when A is an $n \times n$ matrix. Also a matrix is invertible if and only if all its eigenvalues are non-zero. Clearly, if a matrix has an eigenvalue of zero then it must have a non-trivial nullspace containing the corresponding eigenvectors, which would make the matrix singular, because eigenvectors per definition are non-zero.

From (2) we can deduce that A must only have nonnegative eigenvalues. This is because for any eigenvector v we have $v^T A v = v^T (\lambda v) = \lambda (v^T v)$ and since $v^T v > 0$, because v is an eigenvector, and $v^T A v \geq 0$ then $\lambda (v^T v) \geq 0$ and thus $\lambda \geq 0$. So since all eigenvalues of a psd matrix is non-zero, psd matrices aren't necessarily non-singular. However, if we add a positive multiple of the identity matrix to A , $A + \mu I$, $\mu > 0$, we get a positive definite matrix. This is because for any eigenvector v we have $(A + \mu I)v = Av + \mu v = \lambda v + \mu v = (\lambda + \mu)v$, so if λ is an eigenvalue of A then $\lambda + \mu$ is an eigenvalue of $A + \mu I$. Since A is psd, we know $\lambda \geq 0$, and thus $\lambda + \mu > 0$ for all eigenvalues. Therefore, $A + \mu I$ has only positive eigenvalues and is consequently positive definite and invertible.

The symmetry of A leads to two crucial characteristics that guarantee the existence of an eigenvalue decomposition. Firstly, all eigenvalues of a symmetric matrix are real. To see this, consider an eigenvalue λ of A with corresponding eigenvector v . Since A is symmetric, using the properties of inner products we get

$$\lambda(v, v) = (\lambda v, v) = (Av, v) = (v, A^*v) = (v, Av) = \bar{\lambda}(v, v).$$

Because v is an eigenvector $v^*v \neq 0$ so $\lambda = \bar{\lambda}$, which implies that all eigenvalues are real.

Secondly, eigenvectors corresponding to distinct eigenvalues are orthogonal. Consider two eigenvectors v_1 and v_2 corresponding to distinct eigenvalues λ_1 and λ_2 . We have

$$\lambda_1(v_1, v_2) = (\lambda_1 v_1, v_2) = (Av_1, v_2) = (v_1, A^*v_2) = (v_1, Av_2) = (v_1, \lambda_2 v_2) = \bar{\lambda}_2(v_1, v_2).$$

Because the eigenvalues are real we can rearrange this to get

$$(\lambda_1 - \lambda_2)(v_1, v_2) = 0.$$

Since $\lambda_1 \neq \lambda_2$, it follows that $(v_1, v_2) = 0$ and they are thus orthogonal.

From the fundamental theorem of algebra, the characteristic polynomial of A has n roots in $\mathbb{C}$ (We will return to the characteristic polynomial in the section about the Cayley-Hamilton theorem). Since all eigenvalues are real, these roots are real. Moreover, each eigenvalue λ has two important multiplicities: the algebraic multiplicity (its multiplicity as a root of the characteristic polynomial) and the geometric multiplicity (the dimension of its eigenspace).

For a general matrix, it is possible for the geometric multiplicity to be strictly less than the algebraic multiplicity. However, for symmetric matrices, the geometric multiplicity equals the algebraic multiplicity for each eigenvalue and we can therefore use the Gram-Schmidt algorithm to get n orthogonal eigenvectors.

Having established both that the eigenvalues are real and that the algebraic and geometric multiplicities coincide, we conclude that a symmetric A admits a complete basis of eigenvectors. After normalizing these eigenvectors to have unit length, we arrange them as columns in an orthogonal matrix U . Placing the corresponding eigenvalues on the diagonal of a matrix Λ , we obtain the eigenvalue decomposition

$$A = U\Lambda U^T,$$

where Λ is a diagonal matrix comprising the eigenvalues of A and U is an orthogonal matrix containing the corresponding eigenvectors. This decomposition is guaranteed to exist for all symmetric matrices, unlike general (non-symmetric) matrices which may fail to have a complete set of eigenvectors. [5]

2.4 Conjugate Gradient Method

Let $A \in \mathbb{R}^{n \times n}$ be a symmetric positive definite matrix and consider the linear system of equations

$$Ax = b. \quad (3)$$

If we want to find the solution x which satisfies (3) one obvious approach would be by using direct methods such as performing a Cholesky decomposition or Gaussian elimination and then performing back substitution. While memory and computational constraints are rarely a limiting factor for small to medium-sized systems, the $O(n^3)$ computational complexity (required arithmetic operations) of factorization quickly grows to impractical sizes for larger matrices. Furthermore, if A is a sparse matrix, direct methods may suffer from fill-in during factorization, where the sparse matrix gets factored into dense matrices, significantly increasing both computational cost and memory requirements. This is where the conjugate gradient method comes into play, which is an iterative method used for solving systems of the form (3) relying only on matrix vector products. As an iterative method, it progressively improves the accuracy of solutions with each iteration, enabling control over the solution precision through the number of iterations performed.[6] Since the conjugate gradient method doesn't need to factorize A , it completely circumvents the problem of fill-in during factorization of sparse matrices. Generally speaking, if the matrix vector products are cheap to calculate or if the system matrix has a structure which makes the conjugate gradient method converge faster then the conjugate gradient method is desirable in comparison to direct methods.

Since A is assumed to be symmetric and positive definite the problem of solving (3) is equivalent to minimizing the quadratic function given by the expression

$$f(x) = \frac{1}{2}x^T Ax - b^T x + c$$

where c is a constant. This can be expressed equivalently as

$$f(x) = \frac{1}{2}\|x - A^{-1}b\|_A^2 - \frac{1}{2}\|A^{-1}b\|_A^2 \quad (4)$$

using the A -norm $\|v\|_A$ defined as $\|v\|_A = \sqrt{v^T A v}$ for any vector $v \in \mathbb{R}^n$.

In order to find this minimum, the CG (conjugate gradient) method generates a sequence of approximate solutions $x_{(k)}$, residuals $r_{(k)} = b - Ax_{(k)}$ (which measure how far we are from satisfying the original equation), and search directions $d_{(k)}$. The key idea is that these search directions are A -conjugate, meaning $d_{(i)}^T A d_{(j)} = 0$ for $i \neq j$. This property allows CG to converge in at most n iterations in exact arithmetic, where n is the size of the matrix A . In practice, CG often converges to a good approximation in far fewer than n iterations.

The algorithm proceeds as follows:

1. Initialization: Choose an initial guess $x_{(0)}$ and set

$$d_{(0)} = r_{(0)} = b - Ax_{(0)}$$

2. For $i = 0, 1, 2, \dots$ until some stopping criterion is satisfied:

$$\alpha_i = \frac{r_{(i)}^T r_{(i)}}{d_{(i)}^T A d_{(i)}}$$

$$x_{(i+1)} = x_{(i)} + \alpha_i d_{(i)}$$

$$r_{(i+1)} = r_{(i)} - \alpha_i A d_{(i)}$$

$$\beta_i = \frac{r_{(i+1)}^T r_{(i+1)}}{r_{(i)}^T r_{(i)}}$$

$$d_{(i+1)} = r_{(i+1)} + \beta_i d_{(i)}$$

Here, α_i is chosen to minimize $f(x)$ along the search direction $d_{(i)}$, making it the optimal step size along this direction, and β_i is used to ensure the A -conjugacy of the search directions.

One of the major advantages of the CG method is that it only requires matrix-vector products with A , rather than explicit storage or factorization of A . This makes it particularly suitable for large, sparse systems where A is only available as an operator[1]. Moreover, since CG often achieves good approximations well before n iterations, it can be particularly efficient when an exact solution is not required.

2.5 Nyström Approximation

Very large matrices often arise in the fields of data analysis and scientific computing. Such matrices of high order are computationally expensive to store and access in its entirety so an approximation, that allows for a cheaper representation, is crucial. To this end the Nyström approximation was developed, which is a formula used to approximate high order matrices using low rank matrices.

Let the target matrix $A \in \mathbb{R}^{n \times n}$ be an arbitrary symmetric psd matrix. Then the Nyström approximation of A is given by the equation

$$\hat{A}_{\text{nys}} = (A\Omega)(\Omega^T A \Omega)^\dagger (A\Omega)^T \quad (5)$$

where $\dagger$ denotes the Moore-Penrose pseudo inverse and $\Omega \in \mathbb{R}^{n \times \ell}$ determines the kind of rank ℓ Nyström approximation we want. So if Ω is a Gaussian matrix, that is a matrix whose entries are independently drawn from a standard normal Gaussian distribution, then we get the *randomized Nyström approximation*. It offers a cheap and efficient positive semi-definite approximation of A , with the range of A and the sketch $A\Omega$ being aligned. The use of randomness in its construction ensures that $\hat{A}$ is a close approximation to the original matrix A with high probability [7]. However, if Ω is comprised of unique standard basis vectors then we get the *column Nyström approximation* with respect to the standard basis vectors chosen. I.e. if we let $\Omega = [e_1 \ e_3]$ then we will get the Nyström approximation using the first and third column of A .

The *column Nyström approximation* has the following important properties:

1. The *column Nyström approximation* $\hat{A}$ agrees with the target matrix A in the distinguished columns. That is, $\hat{A}\Omega = A\Omega$.
2. The range of the *column Nyström approximation* $\hat{A}$ coincides with the span of the distinguished columns: $\text{range}(\hat{A}) = \text{range}(A\Omega)$.
3. Both the *column Nyström approximation* $\hat{A}$ and the difference between the target matrix and the *column Nyström approximation* $A - \hat{A}$ are psd matrices.

In fact, $\hat{A} - B$ will always be psd for all psd matrices $B \in \mathbb{R}^{n \times \ell}$ which satisfies property 2. and 3.

The following two sections about the Cayley-Hamilton Theorem and Krylov Subspaces are needed later for proving an important property of the convergence of the conjugate gradient method.

2.6 Cayley-Hamilton Theorem

We start by recalling that the characteristic polynomial of a matrix $A \in \mathbb{R}^{n \times n}$ is defined as the monic polynomial

$$\chi_A(\lambda) = \det(\lambda I - A) = c_0 + c_1\lambda + \cdots + c_{n-1}\lambda^{n-1} + \lambda^n.$$

Since the eigenvalues of A are roots of χ_A , it is possible express the coefficients $c_0, \dots, c_{n-1}$ in terms of signed sums of the eigenvalues according to Vieta's formula[8]. We also know from the *Cayley-Hamilton theorem* that χ_A is an *annihilating polynomial* for A , i.e.,

$$\chi_A(A) = c_0I + c_1A + \cdots + c_{n-1}A^{n-1} + A^n = 0. \quad (6)$$

Also, the *minimal polynomial* of A is defined as the monic polynomial μ_A of least degree such that $\mu_A(A) = 0$. By the polynomial long division we can write $f(x) = q(x)\mu_A(x) + r(x)$ where $\deg(r(x)) < \deg(\mu_A(x))$ and $f(x)$ is any other annihilating polynomial of A . Since both $f(A) = 0$ and $\mu_A(A) = 0$, we have $r(A) = 0$ and thus the minimal polynomial must divide any annihilating polynomial. Furthermore, for any eigenvalue λ of A with corresponding

eigenvector v we have $\mu_A(A)v = \mu_A(\lambda)v = 0$, which implies that $\mu_A(\lambda) = 0$ since $v \neq 0$. Therefore the minimal polynomial must have all eigenvalues as roots so if A has distinct eigenvalues then the minimal polynomial of A is the same as the characteristic polynomial of A .

Noting that $c_0 = \chi_A(0) = \det(-A) = (-1)^n \det(A)$, we see that $c_0 \neq 0$ if and only if A is nonsingular that is, A^{-1} exists. If we multiply equation 6 by A^{-1} we get

$$\begin{aligned}\chi_A(A)A^{-1} &= c_0A^{-1} + c_1I + \dots + c_{n-1}A^{n-2} + A^{n-1} = 0 \\ \implies A^{-1} &= -\frac{c_1}{c_0}I - \dots - \frac{c_{n-1}}{c_0}A^{n-2} - \frac{1}{c_0}A^{n-1}\end{aligned}$$

i.e., A^{-1} can be expressed as a polynomial in A of degree $n-1$. Similarly, by multiplying with $b \in \mathbb{R}^n$,

$$\chi_A(A)A^{-1}b = 0 \iff A^{-1}b = -\frac{c_1}{c_0}b - \dots - \frac{c_{n-1}}{c_0}A^{n-2}b - \frac{1}{c_0}A^{n-1}b,$$

we see that

$$A^{-1}b \in \text{span}\{b, Ab, \dots, A^{n-1}b\}, \quad (7)$$

which finally leads us to the definition of Krylov subspaces which are subspaces of the form $\text{span}\{b, Ab, \dots, A^{n-1}b\}$.

2.7 Krylov Subspaces

Let $A \in \mathbb{R}^{n \times n}$ and $b \in \mathbb{R}^n \setminus \{0\}$, then the Krylov subspace of order m generated by A and b is defined as

$$\mathcal{K}_m(A, b) = \text{span}\{b, Ab, \dots, A^{m-1}b\}, \quad m \in \mathbb{N}.$$

Since a polynomial of degree n can be thought of as a linear combination of powers of x up to n we can write

$$\mathcal{K}_m(A, b) = \{p(A)b \mid p \in \mathcal{P}_{m-1}\}, \quad (8)$$

where $\mathcal{P}_{m-1}$ denotes the set of polynomials of degree at most $m-1$. We also note that any Krylov subspace is a subspace of the subsequent Krylov Subspace i.e.

$$\mathcal{K}_m(A, b) \subseteq \mathcal{K}_{m+1}(A, b), \quad m \in \mathbb{N}.$$

Furthermore since the dimension of the Krylov subspace is bounded by the number of vectors spanning the subspace (m), the dimension of the space of which A and b resides in (n) and the dimension of the subspace spanned by $Ab, \dots, A^{m-1}b$ plus an extra dimension accounting for b ($\text{rank}(A)+1$), we have that $\dim \mathcal{K}_m(A, b) \leq \min\{m, n, \text{rank}(A)+1\}$. Clearly we have that $\mathcal{K}_{m+1}(A, b) = \mathcal{K}_m(A, b)$ if and only if $A^m b \in \mathcal{K}_m(A, b)$. But if $A^m b \in \mathcal{K}_m(A, b)$ then we can write

$$A^m b = c_0 b + c_1 Ab + \dots + c_{m-1} A^{m-1} b$$

and by multiplying this equation by A on the left side we get

$$A^{m+1}b = c_0Ab + c_1A^2b + \cdots + c_{m-1}A^mb.$$

But since we just established that $A^mb \in \mathcal{K}_m(A, b)$ we must also have that $A^{m+1}b \in \mathcal{K}_m(A, b)$. This argument can be extended to prove that $A^{m+i}b \in \mathcal{K}_m(A, b)$ for all $i \in \mathbb{N}$ and since A was chosen at random we have proved that if a Krylov subspace is a subspace of the previous Krylov subspace then all subsequent Krylov subspaces will be a subspace of that subspace or equivalently

$$A^mb \in \mathcal{K}_m(A, b) \implies A^ib \in \mathcal{K}_m(A, b), \quad i \geq m.$$

If we denote the smallest m such that $A^m \in \mathcal{K}_m(A, b)$ by m_0 then m_0 must be the maximal dimension of Krylov subspace i.e. $m_0 = \dim \text{span}\{b, Ab, A^2b, \dots\}$. Clearly this implies that $\dim \mathcal{K}_m(A, b) = \min(m, m_0)$ and $\{b, Ab, \dots, A^{m_0-1}b\}$ is a basis for the Krylov subspace $\mathcal{K}_{m_0}(A, b)$. Moreover, this subspace exhibits A -invariance, meaning that $Ax \in \mathcal{K}_{m_0}(A, b)$ for all $x \in \mathcal{K}_{m_0}(A, b)$.

2.8 Convergence Analysis of the Conjugate Gradient Method

2.8.1 Condition number

Naturally, the CG method can be expected to converge at different rates for different linear systems of equations $Ax = b$. One factor influencing the difference in convergence can be attributed to the condition number of A . Recall that the condition number, $\kappa(A)$ of A is defined by the expression:

$$\kappa(A) = \|A\| \cdot \|A^{-1}\| \quad (9)$$

where $\|A\|$ denotes the operator norm of A .

The condition number can be interpreted as an upper bound / worst case for the factor which you have to multiply the relative right hand side error with to get the relative solution error. Specifically:

$$\frac{\|\Delta x\|}{\|x\|} \leq \frac{\|\Delta b\|}{\|b\|} \cdot \kappa(A)$$

where Δx is the error in the solution and Δb is the error in b .

Since the operator norm of a matrix equals its largest singular value, (9) can be written equivalently as

$$\kappa(A) = \frac{s_1}{s_n}$$

where s_1 is the largest singular value of A and s_n is the smallest.[5] However because we have assumed that A is symmetric positive definite this can be further simplified to

$$\kappa(A) = \frac{\lambda_{\max}(A)}{\lambda_{\min}(A)}, \quad (10)$$

where $\lambda_{\max}$ is the maximum eigenvalue and $\lambda_{\min}$ is the minimum eigenvalue.

Thus the rate of convergence of the CG method can be related equation (10). The exact relation is provided in Equation 52 of Shewchuk [1]:

$$\|x^{(i)} - x^*\|_A \leq 2 \left(\frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1} \right)^i \|x^{(0)} - x^*\|_A \quad (11)$$

where x^* denotes the true solution. Thus the condition number provides an upper bound of the error of the CG method at the i -th iteration.

2.8.2 Spectrum and right hand side

Consider a sequence $\{x_k\}_{k=0}^n$ in $\mathbb{R}^n$ defined by $x_0 = 0$ and

$$x_k = \arg \min_{x \in \mathcal{K}_k(A, b)} f(x), \quad k \in \mathbb{N},$$

where $f(x)$ is given by (4). Recall from section 2.4, that since A is assumed to be positive symmetric definite, the solution to $Ax = b$ must coincide with the minimizer of $f(x)$. And since $A^{-1}b \in \mathcal{K}_n(A, b)$ by (7), we can conclude that $x_n = A^{-1}b$. Furthermore, from (8) we have that $x_k = p(A)b$ for some polynomial $p \in \mathcal{P}^{k-1}$, and thus

$$f(x_k) - f(A^{-1}b) = \inf_{x \in \mathcal{K}_k(A, b)} \frac{1}{2} \|x - A^{-1}b\|_A^2 - 0 = \inf_{p \in \mathcal{P}_{k-1}} \frac{1}{2} \|(p(A) - A^{-1})b\|_A^2,$$

where we in the second equality used the fact that x_k can be expressed with the polynomial which minimizes $f(x)$. Utilizing the symmetry of A it is possible to perform an eigenvalue

decomposition $A = Q\Lambda Q^T$ yielding

$$\begin{aligned}
f(x_k) - f(A^{-1}b) &= \inf_{p \in \mathcal{P}_{k-1}} \frac{1}{2} \| (p(Q\Lambda Q^T) - (Q\Lambda Q^T)^{-1})b \|_A^2 \\
&= \inf_{p \in \mathcal{P}_{k-1}} \frac{1}{2} \| (Qp(\Lambda)Q^T - Q\Lambda^{-1}Q^T)b \|_A^2 \\
&= \inf_{p \in \mathcal{P}_{k-1}} \frac{1}{2} \| Q(p(\Lambda) - \Lambda^{-1})Q^Tb \|_A^2 \\
&= \inf_{p \in \mathcal{P}_{k-1}} \frac{1}{2} (Q(p(\Lambda) - \Lambda^{-1})Q^Tb)^T Q\Lambda Q^T (Q(p(\Lambda) - \Lambda^{-1})Q^Tb) \\
&= \inf_{p \in \mathcal{P}_{k-1}} \frac{1}{2} b^T Q(p(\Lambda) - \Lambda^{-1})^T \Lambda (p(\Lambda) - \Lambda^{-1})Q^Tb \\
&= \inf_{p \in \mathcal{P}_{k-1}} \frac{1}{2} \sum_{i=1}^n b^T q_i (p(\lambda_i) - \frac{1}{\lambda_i}) \lambda_i (p(\lambda_i) - \frac{1}{\lambda_i}) q_i^T b \\
&= \inf_{p \in \mathcal{P}_{k-1}} \frac{1}{2} \sum_{i=1}^n \lambda_i (p(\lambda_i) - \frac{1}{\lambda_i})^2 (q_i^T b)^2 \\
&= \inf_{p \in \mathcal{P}_{k-1}} \frac{1}{2} \sum_{i=1}^n \lambda_i \left(\frac{\lambda_i p(\lambda_i) - 1}{\lambda_i} \right)^2 (q_i^T b)^2 \\
&= \inf_{p \in \mathcal{P}_{k-1}} \frac{1}{2} \sum_{i=1}^n \lambda_i \left(\frac{1 - \lambda_i p(\lambda_i)}{\lambda_i} \right)^2 (q_i^T b)^2 \\
&= \inf_{p \in \mathcal{P}_{k-1}} \frac{1}{2} \sum_{i=1}^n \frac{(1 - \lambda_i p(\lambda_i))^2}{\lambda_i} (q_i^T b)^2. \tag{12}
\end{aligned}$$

where we in the second and ninth equality used the property that $p(Q\Lambda Q^T) = Qp(\Lambda)Q^T$ and $(a - b)^2 = (b - a)^2$ respectively. Noting that $\tilde{p}(\lambda) = 1 - \lambda p(\lambda)$ is a polynomial of degree at most k , we can express the infimum in (12) as

$$f(x_k) - f(A^{-1}b) = \inf_{\tilde{p} \in \mathcal{P}_k} \left\{ \frac{1}{2} \sum_{i=1}^n \frac{\tilde{p}(\lambda_i)^2}{\lambda_i} (q_i^T b)^2 \mid \tilde{p}(0) = 1 \right\}. \tag{13}$$

Now, if A has at most k distinct eigenvalues, then the sum in equation (13) will at most contain k terms, because any repeated eigenvalues can be grouped together by factoring them out of the $\frac{1}{\lambda} (q_i^T b)^2$ terms. Similarly if b is a linear combination of eigenvectors corresponding to at most k distinct eigenvalues, that is $b = c_1 q_{i_1} + c_2 q_{i_2} + \dots + c_k q_{i_k}$, then $q_j^T b$ will be zero for all eigenvectors q_j not in this linear combination. This is because A is assumed to have a spectral decomposition (A is symmetric) so $\{q_1, q_2, \dots, q_n\}$ comprises an orthonormal basis of eigenvectors. This again reduces the sum to at most k non-zero terms. Since $\tilde{p}$ is a polynomial of degree k we can construct it so that $p(\lambda_i) = 0$ for all k distinct eigenvalues because $\lambda_i \neq 0$ per assumption. Thus, x_k is a minimizer of $f(x)$ over $\mathbb{R}^n$ if A has at most k distinct eigenvalues or if b is a linear combination of the eigenvectors of A corresponding

to at most k distinct eigenvalues. In other words we can conclude that if A has k distinct eigenvalues then the conjugate gradient method will converge in at most k iterations. We can also deduce that it will converge in at most j iterations if b is in the span of eigenvectors corresponding to j distinct eigenvalues of A . This shows that the full picture of convergence is a bit more convoluted than just a question of either having a high or low condition number. Rather, the spectrum of A and the choice of the right side b also plays a crucial role. [9]

2.9 Preconditioning

Referring to equation (11) we know that the condition number provides an upper bound for the convergence of the CG method. We also know from the discussion in the previous section that the convergence of the CG method is bound by the number of distinct eigenvalues of A . Utilizing this information we wish to somehow implement a method which improves the convergence rate of the CG method for any given system $Ax = b$. This method should work by somehow either lowering the condition number A or better clustering its' eigenvalues, that is increasing the multiplicities of eigenvalues, or a combination of both. To this end we introduce the concept of preconditioning which involves solving the modified system

$$M^{-1}Ax = M^{-1}b, \quad (14)$$

where M is called the preconditioner which is a symmetric positive definite matrix that is cheaper to invert than A . The reason why M is chosen to be cheaper to invert than A is because in each iteration of preconditioned CG, we need to compute $M^{-1}r$. Clearly we also wish to choose M such that either $\kappa(M^{-1}A) < \kappa(A)$ or such that the eigenvalues of $M^{-1}A$ becomes better clustered than those of A and thus obtaining a system which converges faster using the CG method.

When preconditioning, we encounter a fundamental difficulty: even when both M and A are symmetric and positive-definite, $M^{-1}A$ is generally neither symmetric nor positive-definite. However, we can resolve this through a mathematical transformation. For any symmetric, positive-definite matrix M , there exists a matrix E (not necessarily unique) such that $EE^T = M$. We can obtain such an E through Cholesky factorization. A crucial property is that $M^{-1}A$ and $E^{-1}AE^{-T}$ share the same eigenvalues and thus also the same beneficial convergence properties. We can prove this by showing that if v is an eigenvector of $M^{-1}A$ with eigenvalue λ , then $E^T v$ must be an eigenvector of $E^{-1}AE^{-T}$ with the same eigenvalue λ :

$$\begin{aligned} \lambda E^T v &= E^T M^{-1} A v \\ &= E^T (E^{-T} E^{-1}) A v \\ &= (E^T E^{-T}) E^{-1} A v \\ &= E^{-1} A v \\ &= E^{-1} A (E^{-T} E^T) v \\ &= (E^{-1} A E^{-T}) (E^T v) \end{aligned}$$

This allows us to transform our original system $Ax = b$ into:

$$E^{-1}AE^{-T}\hat{x} = E^{-1}b, \quad \text{where} \quad \hat{x} = E^Tx$$

Since $E^{-1}AE^{-T}$ is symmetric and positive-definite, we can solve for $\hat{x}$ using the CG method, and then recover x from the transformed system.

Applying the preconditioner directly to the CG method from section 2.3 yields the algorithm

$$\begin{aligned} \tilde{d}_{(0)} &= \tilde{r}_{(0)} = E^{-1}b - E^{-1}AE^{-T}\tilde{x}_{(0)}, \\ \alpha_{(i)} &= \frac{\tilde{r}_{(i)}^T \tilde{r}_{(i)}}{\tilde{d}_{(i)}^T E^{-1}AE^{-T} \tilde{d}_{(i)}}, \\ \tilde{x}_{(i+1)} &= \tilde{x}_{(i)} + \alpha_{(i)} \tilde{d}_{(i)}, \\ \tilde{r}_{(i+1)} &= \tilde{r}_{(i)} - \alpha_{(i)} E^{-1}AE^{-T} \tilde{d}_{(i)}, \\ \beta_{(i+1)} &= \frac{\tilde{r}_{(i+1)}^T \tilde{r}_{(i+1)}}{\tilde{r}_{(i)}^T \tilde{r}_{(i)}}, \\ \tilde{d}_{(i+1)} &= \tilde{r}_{(i+1)} + \beta_{(i+1)} \tilde{d}_{(i)}. \end{aligned}$$

However, if we set $r_{(i)} = E^{-T}r_{(i)}$ and $d_{(i)} = E^T \tilde{d}_{(i)}$ and use the identities $\tilde{x}_{(i)} = E^T x_{(i)}$ and $E^{-1}E^{-1} = M^{-1}$, we get the following optimal algorithm:

$$\begin{aligned} r_{(0)} &= b - Ax_{(0)}, \\ d_{(0)} &= M^{-1}r_{(0)}, \\ \alpha_{(i)} &= \frac{r_{(i)}^T M^{-1}r_{(i)}}{d_{(i)}^T A d_{(i)}}, \\ x_{(i+1)} &= x_{(i)} + \alpha_{(i)} d_{(i)}, \\ r_{(i+1)} &= r_{(i)} - \alpha_{(i)} A d_{(i)}, \\ \beta_{(i+1)} &= \frac{r_{(i+1)}^T M^{-1}r_{(i+1)}}{r_{(i)}^T M^{-1}r_{(i)}}, \\ d_{(i+1)} &= M^{-1}r_{(i+1)} + \beta_{(i+1)} d_{(i)}. \end{aligned}$$

This method benefits from neither having to compute E nor solving for x in $\hat{x} = E^T x$ at the end.

The effectiveness of a preconditioner M depends primarily on two factors: the condition number of $M^{-1}A$ and how well its eigenvalues are clustered. The practical challenge lies in finding a preconditioner that sufficiently approximates A to improve convergence while keeping the computational cost of calculating $M^{-1}r_{(i)}$ manageable in each iteration. It's worth noting that we don't need to explicitly compute M or M^{-1} rather, we only need to be able to compute the effect of applying M^{-1} to a vector. Within these constraints, there exists a lot of different possible preconditioners each with their own strengths and weaknesses.

From a geometric perspective, preconditioning can be viewed as a transformation that reshapes the contour lines of the quadratic form. Recall that the unpreconditioned quadratic form is given by:

$$f(x) = \frac{1}{2}x^T Ax - b^T x$$

whose contour lines form ellipses (in 2D) or ellipsoids (in higher dimensions) when A is symmetric positive definite. The shape of these ellipsoids is determined by the eigenvalues and eigenvectors of A - the eigenvectors determine the principal directions of the ellipsoids, while the eigenvalues determine how much the ellipsoids are stretched along these directions. The theoretical ideal would be $M = A$, which would yield a condition number of one for $M^{-1}A$ and create a perfectly spherical quadratic form, leading to convergence in a single iteration. However, this is impractical since the preconditioning step would require solving the original system $Mx = b$ which defeats the purpose of using the CG method.

The simplest preconditioner one can use is diagonal preconditioning also known as Jacobi preconditioning, where M is constructed as a diagonal matrix sharing the diagonal entries of A . This preconditioner aims to improve convergence by decreasing the condition number of A of the original system of equations $Ax = b$. It does this by effectively scaling the quadratic form along the coordinate axes and thus obtaining more spherical contour lines. While a diagonal matrix is very cheap to invert, this preconditioning technique usually only provides moderate improvement in convergence. [1]

2.10 Randomized Nyström Preconditioning

The core idea of randomized Nyström Preconditioning is to construct a preconditioner based on a low-rank approximation of the matrix A using the randomized Nyström method for which, when applied to our linear system, dramatically reduces the condition number. Consider the regularized linear system:

$$(A + \mu I)x = b, \tag{15}$$

where $A \in \mathbb{R}^{n \times n}$ is a symmetric positive semidefinite (psd) matrix and $\mu \geq 0$ is a regularization parameter added to the matrix to ensure numerical stability and improve the condition number. This type of linear system often arises when solving a regularized least-squares problem. Recall that the randomized Nyström approximation of A with desired rank ℓ takes the form

$$\hat{A}_{\text{nys}} = (A\Omega)(\Omega^T A\Omega)^\dagger (A\Omega)^T.$$

Naturally the rank ℓ of the randomized Nyström approximation determines the quality of the preconditioner with higher values of ℓ capturing more information of A and thus providing better preconditioners. The choice of ℓ involves a trade-off: larger values improve the approximation quality but increase both computational costs and storage requirements. In practice, empirical evaluations are typically used to determine an optimal ℓ that balances these competing factors. With the eigenvalue decomposition $\hat{A}_{\text{nys}} = U\hat{\Lambda}U^T$ of the randomized Nyström approximation, we can construct the Nyström preconditioner [10]:

$$P = \frac{1}{\hat{\lambda}_\ell + \mu} U(\hat{\Lambda} + \mu I)U^T + (I - UU^T)$$

which is a preconditioner for the regularized linear system (15). Here, $\hat{\lambda}_\ell$ denotes the smallest eigenvalue from the diagonal matrix $\hat{\Lambda}$ in the Nyström approximation's eigendecomposition, and μ is the regularization parameter. The inverse of P which is used to implement the preconditioner is given by the formula

$$P^{-1} = (\hat{\lambda}_\ell + \mu)U(\hat{\Lambda} + \mu I)^{-1}U^T + (I - UU^T). \quad (16)$$

Before implementing randomized Nyström preconditioning for the CG method we first need an implementation of the randomized Nyström approximation:

Algorithm 1 Randomized Nyström Approximation

Input: Symmetric psd matrix $A \in \mathbb{R}^{n \times n}$ and rank ℓ

Output: Nyström approximation in factored form $\hat{A}_{nys} = U\hat{\Lambda}U^T$

- | | |
|--|------------------------------|
| 1: $\Omega = \text{randn}(n, \ell)$ | ▷ Gaussian test matrix |
| 2: $\Omega = \text{qr}(\Omega, 0)$ | ▷ Thin QR decomposition |
| 3: $Y = A\Omega$ | ▷ ℓ matvecs with A |
| 4: $\nu = \text{eps}(\text{norm}(Y, \text{'fro'})$ | ▷ Compute shift |
| 5: $Y_\nu = Y + \nu\Omega$ | ▷ Shift for stability |
| 6: $C = \text{chol}(\Omega^T Y_\nu)$ | |
| 7: $B = Y_\nu / C$ | |
| 8: $[U, \Sigma, \sim] = \text{SVD}(B, 0)$ | ▷ Thin SVD |
| 9: $\hat{\Lambda} = \max\{0, \Sigma^2 - \nu I\}$ | ▷ Remove shift, compute eigs |
-

Line 1-3 are used for computing the sketch matrix $Y = A\Omega$, $Y \in \mathbb{R}^{n \times \ell}$ with rank ℓ equal to the rank of $\hat{A}_{nys}$. The reason for the QR decomposition on line 2 is to ensure that the columns of Ω are orthonormal. Line 4-6 serve the purpose of making the computations numerically stable. Since the SVD and eigendecompositions coincide for symmetric positive semi-definite matrices, we perform thin SVD on line 8 to get Σ containing the ℓ largest singular values, and U comprising the corresponding left singular vectors. Finally line 9 ensures that $\hat{\Lambda}$ doesn't contain negative elements.

We are now ready to present the implementation of Nyström PCG (Randomized Nyström Preconditioned Conjugate Gradient Method):

Algorithm 2 Nyström PCG

Input: Psd matrix A , right-hand side b , initial guess x_0 , regularization parameter μ , sketch size ℓ and solution tolerance η

Output: Approximate solution $\hat{x}$ to regularized system (1.1)

```

1:  $[U, \tilde{\Lambda}] = \text{RandomizedNyströmApproximation}(A, \ell)$  ▷ using alg. 1
2:  $r_0 = b - (A + \mu I)x_0$ 
3:  $z_0 = P^{-1}r_0$  ▷ Using (16)
4:  $p_0 = z_0$ 
5: while  $\|r\|_2 > \eta$  do
6:    $v = (A + \mu I)p_0$  ▷ compute step size
7:    $\alpha = \frac{r_0^\top z_0}{p_0^\top v_0}$  ▷ compute step size
8:    $x = x_0 + \alpha p_0$  ▷ update solution
9:    $r = r_0 - \alpha v$  ▷ update residual
10:   $z = P^{-1}r$  ▷ find search direction via (16)
11:   $\beta = \frac{r^\top z}{r_0^\top z_0}$  ▷ find search direction via (16)
12:   $x_0 \leftarrow x, r_0 \leftarrow r, p_0 \leftarrow z + \beta p_0, z_0 \leftarrow z$ 
13: end while

```

Comparing this with the algorithm from Section 2.8, we can see they are essentially the same - the only difference is that we're applying the algorithm to $(A + \mu I)x = b$ instead of $Ax = b$ and have changed some of the variables.

2.11 Pivoted partial cholesky algorithm

The pivoted partial cholesky algorithm is given by the pseudocode below:

Algorithm 3 Pivoted Partial Cholesky Algorithm

Input: Psd matrix $A \in \mathbb{R}^{n \times n}$ and approximation rank ℓ

Output: Pivot set $S = \{s_1, \dots, s_k\}$ and matrix $F \in \mathbb{R}^{n \times \ell}$ defining Nyström approximation $\hat{A} = FF^T$

```

1: Initialize  $F \leftarrow 0_{n \times \ell}$ 
2: for  $i = 1$  to  $\ell$  do
3:   Select a pivot  $s_i \in \{1, \dots, N\}$ 
4:    $g \leftarrow A_{s_i}$  ▷ Evaluate the  $s_i$ -th column of the matrix  $A$ 
5:    $g \leftarrow g - F_{:,1:(i-1)} F_{s_i,1:(i-1)}^T$  ▷ Remove the influence of previously chosen columns
6:    $F_{:,i} \leftarrow g / \sqrt{g_{s_i}}$ 
7: end for

```

This algorithm provides an efficient method for computing the partial Cholesky decomposition of a rank- ℓ column Nyström approximation of a given symmetric positive semi-definite matrix A . More concise, provided the pivot set $S = \{s_1, \dots, s_k\}$ the algorithm returns the column Nyström approximation $\hat{A} = A_{:,S} A_{S,S}^\dagger A_{S,:}$. An overview of how the algorithm works now follows: The algorithm is first initialized by letting the approximation

$\hat{A}^{(0)} = 0$ and the residual $A^{(0)} = A$. Then for every iteration $i = 1, 2, 3, \dots, k$, we select a new column index $s_i \in \{1, \dots, N\}$ using some pivot rule which we will come back to later. Then we evaluate the s_i column $A_{:,s_i}^{(i-1)}$ of the current residual and use it to calculate the new approximation and the residual:

$$\begin{aligned}\hat{A}^{(i)} &= \hat{A}^{(i-1)} + \frac{A_{:,s_i}^{(i-1)} A_{s_i,:}^{(i-1)}}{A_{s_i,s_i}^{(i-1)}}; \\ A^{(i)} &= A^{(i-1)} - \frac{A_{:,s_i}^{(i-1)} A_{s_i,:}^{(i-1)}}{A_{s_i,s_i}^{(i-1)}}.\end{aligned}$$

Furthermore we can track the diagonal elements of the residual $A^{(i)}$ by using the formula

$$\text{diag}(A^{(i)}) = \text{diag}(A^{(i-1)}) - \frac{1}{A_{s_i,s_i}^{(i-1)}} \cdot |A_{:,s_i}^{(i-1)}|^2. \quad (17)$$

Here, the function $|\cdot|^2 : \mathbb{R}^N \rightarrow \mathbb{R}_+^N$ maps a vector in $\mathbb{R}^N$ to a vector in $\mathbb{R}_+^N$, where each element is squared. It is possible to use equation (17) to track the convergence of the algorithm and then making it stop at a desired accuracy.

The algorithm computes the partial Cholesky decomposition $\hat{A} = FF^T$ by sequentially generating the columns of F . Specifically, the i -th column $F_{:,i}$ is obtained by evaluating the s_i -th column $A_{:,s_i}$ of the input matrix and subtracting the contributions of the previously selected columns.

Thus, at each iteration, $\hat{A}^{(i)}$ represents the Nyström approximation of A based on the columns indexed by $\{s_1, \dots, s_i\}$.

Regarding the computational cost of the Pivoted Partial Cholesky Algorithm, for k iterations, the algorithm evaluates kN entries of the input matrix A . The algorithm uses $\mathcal{O}(k^2N)$ arithmetic operations, while the storage requirement is $\mathcal{O}(kN)$.

In practice, the algorithm is normally run until the trace-norm error satisfies the inequality $\text{tr}(A - \hat{A}) \leq \eta \cdot \text{tr}(A)$. While this adjustment gives an unpredictable number of steps, it effectively controls the error and guarantees that $\hat{A}$ makes for a good approximation.

Now we want to modify the algorithm so that it chooses the optimal set of the pivots which minimizes the trace norm error $\text{tr}(\hat{A} - A)$. To this end we can try some different heuristic pivot selection rules. One naive way to go about this is by choosing a new pivot at random at each iteration with equal probability which is called uniform random pivoting. Another approach is by utilizing the relationship $|a_{ij}| \leq \sqrt{a_{ii}a_{jj}}$ for each $1 \leq i, j \leq N$, which means a large value in the diagonal entry a_{ij} suggests that column j may contain elements of large magnitudes. The rationale of this is that by eliminating such a column from the residual we can hope to substantially reduce the trace of the residual at each step. This idea gives rise to the greedy pivoting rule which is a rule that selects the pivot s_i by identifying the position of the largest diagonal entry in the residual matrix $A^{(i-1)}$. This can be formulated mathematically as

$$s_i \in \arg \max_{1 \leq j \leq N} A_{j,j}^{(i-1)}. \quad (18)$$

where $A_{j,j}^{(i-1)}$ denotes the j -th diagonal entry of the residual matrix $A^{(i-1)}$. However for our experiments we will use RPCholesky pivot selection rule which is a heuristic that combines the idea of the two rules mentioned above. It works by selecting the pivot s_i using an adaptive sampling approach. Specifically the pivots are given by a probability distribution that is directly proportional to the diagonal elements of the current residual matrix, that is

$$\mathbb{P}\{s_i = j\} = \frac{A_{j,j}^{(i-1)}}{\text{tr } A^{(i-1)}} \quad \text{for } j = 1, \dots, N.$$

By incorporating the RPCholesky pivot rule into the Pivoted Partial Cholesky Algorithm we can update Algorithm 3 to get the pseudocode for the RPCholesky algorithm[11]:

Algorithm 4 RPCHOLESKY

Input: Psd matrix $A \in \mathbb{R}^{N \times N}$ and approximation rank ℓ

Output: Pivot set $S = \{s_1, \dots, s_\ell\}$ and matrix $F \in \mathbb{R}^{N \times \ell}$ defining Nyström approximation $\hat{A} = FF^T$

- 1: Initialize $F \leftarrow 0_{N \times \ell}$ and $d \leftarrow \text{diag } A$ ▷ Evaluate diagonal of input matrix
 - 2: **for** $i = 1$ to ℓ **do**
 - 3: Sample pivot $s_i \sim d / \sum_{j=1}^N d_j$ ▷ With probability proportional to residual diagonal
 - 4: $g \leftarrow A(:, s_i)$ ▷ Evaluate column s of input matrix
 - 5: $g \leftarrow g - F(:, 1 : i - 1)F(s_i, 1 : i - 1)^T$ ▷ Remove overlap with previously chosen columns
 - 6: $F(:, i) \leftarrow g / \sqrt{g_{s_i}}$ ▷ Update approximation
 - 7: $d \leftarrow d - |F(:, i)|^2$ ▷ Track diagonal of residual matrix
 - 8: $d \leftarrow \max(d, 0)$ ▷ Ensure diagonal remains nonnegative
 - 9: **end for**
-

But before we can use the RP Cholesky algorithm to construct a preconditioner we first have to transform the partial Cholesky decomposition $\hat{A} = FF^T$ into an eigenvalue decomposition. But this is easily achieved by utilizing the property that given the partial Cholesky decomposition $A = FF^T$ and singular value decomposition $F = USV^T$ we get the corresponding eigenvalue decomposition $A = FF^T = USV^T(USV^T)^T = USV^TVS^TU^T = US^2U^T$. Hence the RPCholesky preconditioned CG method is simply implemented by replacing step 1 of algorithm 2 with the eigenvalue decomposition of the result of algorithm 4.

2.12 Boxplots

Given the probabilistic nature of the conducted experiments, boxplots were used to convey the results making for an effective visualization. This is because boxplots provide a simple yet very informative graphical representation of the distribution of data. However, due to the various ways boxplots can be defined, we will now explain the definition used in our experiment. Naturally, since we used Python's `matplotlib` plotting library to create the boxplots, specifically the `matplotlib.pyplot.boxplot` module, its' definition follows the one stated in the documentation of the module [12]: "The box extends from the first

quartile (Q1) to the third quartile (Q3) of the data, with a line at the median. The whiskers extend from the box to the farthest data point lying within 1.5x the inter-quartile range (IQR) from the box. Flier points are those past the end of the whiskers."

3 Experiments and Results

3.1 Experimental Setup

3.1.1 Problem Definition

The goal of this project is to compare the randomized Nyström preconditioned conjugate gradient method (PCG) with the pivoted partial Cholesky (RPCholesky) preconditioned conjugate gradient method. In order to provide a comprehensive comparison, we will conduct numerical experiments which aim to test their capabilities on real-world examples. Specifically, we will employ both algorithms on two specific test-cases and examine how many iterations are required for the two methods to solve the regularized system of equations $(Ax + \mu I) = b$ within a desired accuracy. The experiments will be run for different choices of the sketch sizes, ℓ of the preconditioners to get an insight of their influence on the results.

3.1.2 Test Matrices

The test matrices chosen for our experiments are the same as those appearing in the RPCholesky article[11]. In particular, they are Gaussian kernel matrices constructed using the formula

$$K_{ij} = e^{-\frac{\|x_i - x_j\|_2^2}{2\sigma^2}}. \quad (19)$$

While the points x_i and x_j can generally belong to any domain space, in our experiments we work with points in $\mathbb{R}^2$. From (19) we can see that the (i, j) -th entry K_{ij} measures the similarity between points x_i and x_j based on their Euclidean distance squared $\|x_i - x_j\|_2^2$. The exponential function maps this distance to a similarity score between 0 and 1, where points that are close together have similarity close to 1, and points far apart have similarity close to 0.

The parameter σ is the kernel bandwidth which controls how quickly the similarity between decreases decays with distance. A larger σ results in points further apart still having significant similarity, while a smaller σ means the similarity drops off more quickly with distance. The resulting matrix K is symmetric positive semi-definite, with entries between 0 and 1, where diagonal entries are always 1 (since $\|x_i - x_i\|_2^2 = 0$) and off-diagonal entries represent the similarity between different points.

We aim to maximize the size of the test matrices, n to reflect realistic applications, where problem sizes are typically large. Direct methods are efficient for small values of n , but as n grows, iterative methods become necessary due to computational and memory constraints. This matches common scenarios in practice, where we often deal with large-scale systems that are impractical to solve using direct methods. Given our computational resources, we

selected $n = 1000$ as it was the largest problem size that could be solved within a reasonable time frame.

The first matrix is constructed using (19) with $N = 10^4$ data points that form a smiling face in $\mathbb{R}^2$. The face is composed of three main elements: two eyes, a parabolic mouth, and a circular outline, all contained within $[-10, 10] \times [-10, 10]$. The kernel bandwidth is set to $\sigma = 2.0$.

The eyes are symmetrically positioned at $(-4, 4)$ and $(4, 4)$, with each eye constructed from $\lceil \sqrt{N} \rceil$ points (equal to 100 points in our case) randomly sampled from the unit disc. The mouth consists of $\lceil N/10 \rceil = 100$ points where the x -coordinates are equispaced on the interval $[-5, 5]$, and each point's y -coordinate is determined by the parabola $y = x^2/16 - 5$. The remaining points (9,700 in our case) are distributed uniformly along a circular boundary of radius 10, forming the face's outline.

The second matrix is constructed using (19) with 1000 points in $\mathbb{R}^2$ that trace a logarithmic spiral defined parametrically by $(e^{0.2t} \cos t, e^{0.2t} \sin t)$. The parameter t consists of 1000 equispaced values that start at $t = 2$ and decrease to $t = 0$, but each value is first raised to the sixth power before being used. This means the original evenly-spaced sequence $[0, 2]$ becomes $[2^6, \dots, 0] = [64, \dots, 0]$, creating denser point distributions in the inner regions of the spiral. The kernel bandwidth is set to $\sigma = 1000$, in order to damp the influence of the outliers which arise as a result of the exponential growth of the spiral's radius. For both test matrices, the right-hand side vector b is generated at random since we aren't investigating its role.

The functions used for constructing the smile and spiral can be found in the code appendix under the name "Test Matrices" with function names `smile` and `robspir` respectively. This is the same code as the one used in the RPCholesky article [13] [14].

3.1.3 Experimental Parameters

Since randomization plays a significant role in the construction of either preconditioner, we will run multiple instances of each experiment to minimize the role of this randomness. To this end we chose to run the experiment 50 times for each choice of the sketch size, as this number strikes a balance for our need for reliable results while keeping the computation time of the experiments under a reasonable threshold. We run each experiment 50 times, where for each of these 50 iterations, new preconditioners are constructed and similarly new right-hand side vectors are generated at random. Also, we set the start guess $x_0 = 0$ since we don't have a rough estimate of the actual solution x . Both preconditioned algorithms are run until reaching a relative residual of 10^{-4} , ensuring a very high accuracy, and we record the number of iterations required to reach this threshold. The convergence criterion is based on the relative residual in the ℓ^2 norm, computed as $\frac{\|b - Ax_k\|_2}{\|b\|_2}$, where x_k is the solution calculated at iteration k . We will run the experiments across the following four different sketch sizes, $\ell = 50, 75, 100$ and 125 . Finally the regularization parameter μ was selected through empirical testing on smaller-scale pilot experiments. Multiple values of μ were evaluated for either matrix using the same parameters as the main experiment but with a matrix size of $n = 100$ to identify the optimal regularization parameters. A balance was sought between maximizing regularization's effect on the improvement of convergence

while keeping μ small enough to avoid over-regularizing the system. This approach resulted in choosing a regularization parameter of 10^{-2} and 10^{-3} for the spiral and smile matrix respectively.

3.2 Implementation Details

The code used for the RPCholesky algorithm is a modified version of the code from the RPCholesky article [13] [14], while the code for the randomized Nyström Approximation and Preconditioned CG method was implemented based on the pseudocode from section 2.9, with assistance from claude.ai. In our implementation of the Preconditioned Conjugate Gradient method, we made one modification to Algorithm 2: we replaced $\|r\|_2$ with $\frac{\|r\|_2}{\|b\|_2}$ in the while loop to measure convergence in terms of relative error.

Furthermore, since the right-hand side vector b is generated at random, a random seed is set in the experiment scripts to ensure that for any choice of the rank parameter ℓ , both methods generate the same sequence of random right-hand side vectors. This not only provides reproducibility but also ensures a fair comparison between Nyström PCG and RPCholesky PCG across all ranks, as both methods are tested on exactly the same sequence of linear systems for any sketch size. Finally, all code implementations are provided in the code appendix with descriptive names indicating their purposes.

3.3 Results

We are now ready to present the results of the experiments which are found in Figure 1 below.

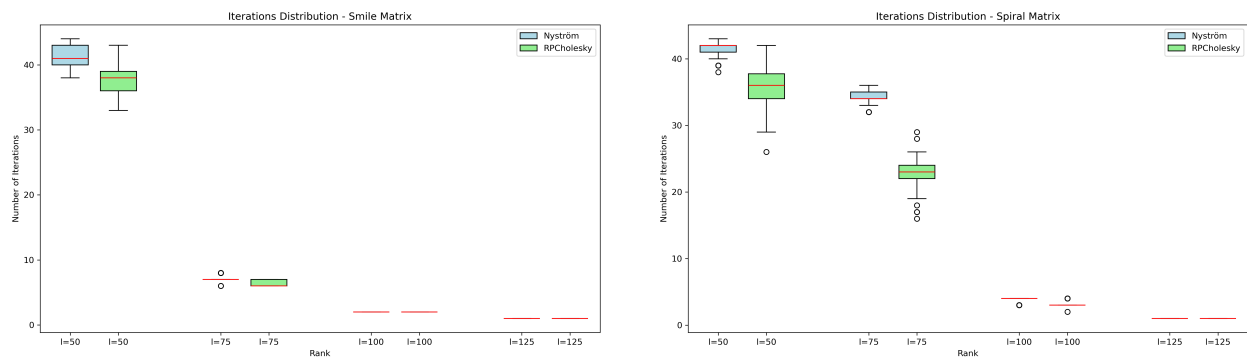


Figure 1: Boxplot distributions of how choosing the rank/sketch size affects the number of iterations required for the randomized Nyström PCG and RPCholesky PCG to converge. Left: smile matrix. Right: spiral matrix.

In order to capture the variability introduced by the random sketching procedures and random right-hand sides, we ran each experiment 50 times and collected statistics on the resulting iteration counts. Figure 1 illustrates these statistics via boxplots, where the central line in each box denotes the median number of iterations, and the box boundaries represent

the 25th and 75th percentiles. The whiskers (and any points lying outside them) highlight outliers that is individual runs that required more iterations to converge than the majority.

It's immediately apparent from just one quick look at the figure that the rate of convergence increases with the sketch size, ℓ . This is the case for either algorithms and system matrix which means that a higher sketch size will generally always lead to faster convergence. This should come as little surprise since a higher sketch size means that more information of the system matrix is utilized and thus a more effective preconditioner is produced. However, this improvement comes with a trade-off: larger sketch sizes incur higher computational costs when forming the preconditioner and thus we have a big overhead associated with the construction of the preconditioner which isn't accounted for in the plots.

Another key observation is that larger sketch sizes also lead to less variability. This observation suggests that a better approximation of the system matrix (whether through randomized Nyström or RPCholesky) yields a more robust preconditioner that consistently get the CG method to converge in fewer iterations. Conversely, when ℓ is small, the variation tends to be greater which suggests that the performance of low rank preconditioners are more influenced by randomness and thus less predictable.

When comparing the two methods, RPCholesky demonstrates a slight but consistent advantage across all sketch sizes for both matrices. While the performance difference is modest, RPCholesky either matches or outperforms the randomized Nyström method in all test cases.

Since algorithm performance in practical applications is measured by CPU time rather than iteration count, we repeated the experiment while tracking CPU time instead of the iteration count. This yielded the following results:

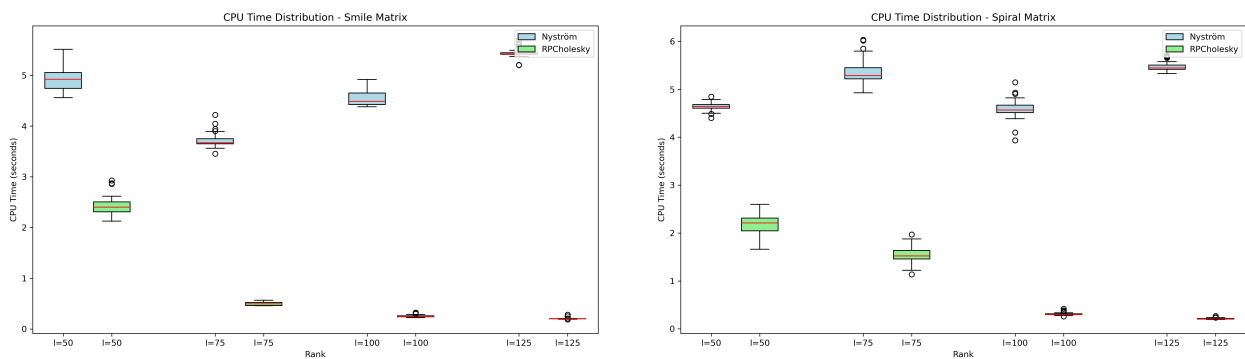


Figure 2: Visualization of how choosing the rank/sketch size affects the elapsed CPU time required for the randomized Nyström PCG and RPCholesky PCG to converge. Left: smile matrix. Right: spiral matrix.

Figure 2 reveals the actual computational workload of running the two algorithms. It shows that the RPCholesky algorithm performs best with the highest possible rank parameter across both test cases, whereas the Nyström preconditioner favors a more modest rank parameter. Specifically, it reaches peak efficiency at $\ell = 50$ for the smile matrix and $\ell = 100$ for the spiral matrix. When compared to Figure 1, Figure 2 paints a very different picture of the Nyström method's performance. While the CPU time for the RPCholesky

method generally follows the same trend as in Figure 1, the performance of the Nyström method decays rapidly with increasing rank, which is quite surprising. This suggests that the computational cost of constructing the Nyström preconditioner significantly outweighs its modest improvement to the CG convergence, resulting in poorer overall performance. To verify this hypothesis, we repeated our experiments while measuring the CPU time required to construct each preconditioner, with results shown in Figure 3.

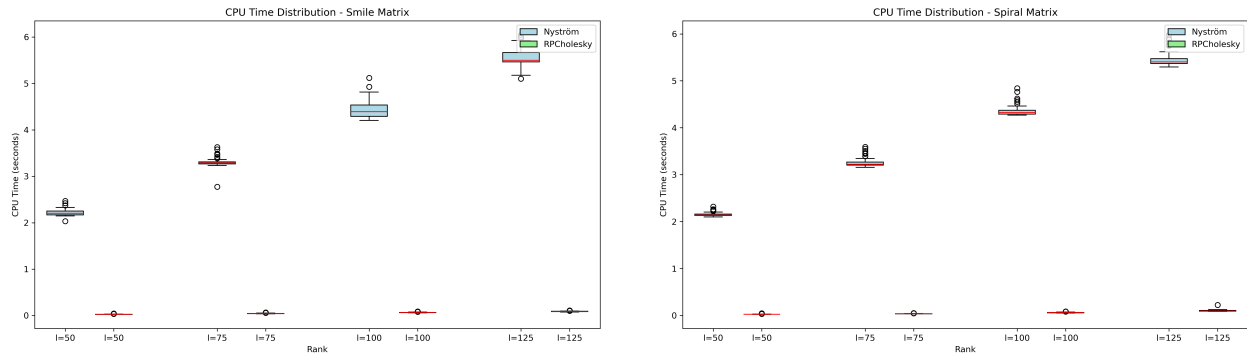


Figure 3: Boxplot distributions of CPU time spent constructing the Randomized Nyström and RPCholesky preconditioners at different rank/sketch sizes. Left: smile matrix. Right: spiral matrix.

This clearly confirms our suspicion regarding the major increase in computational resources required to construct the Nyström preconditioner. The Nyström preconditioner not only starts with significantly higher CPU times at rank $\ell = 50$, but also shows substantial increases with rank size, while the RPCholesky method maintains relatively constant and lower CPU times across all ranks. This contrast is particularly evident in the higher ranks, where the Nyström method's computational overhead becomes markedly larger. Furthermore, this poor scalability holds for both test matrices, suggesting that the Nyström preconditioner is a poor choice for problems involving Gaussian matrices.

To better highlight the CPU time differences that were difficult to discern from the boxplots, we also plotted the histograms of the mean CPU times

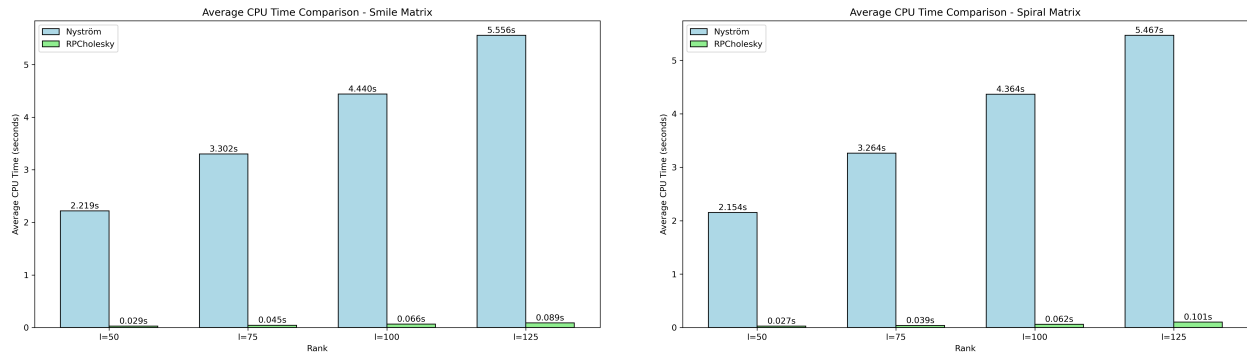


Figure 4: Average CPU time required to construct the Randomized Nyström and RPCholesky preconditioners at various rank/sketch sizes. Each bar represents the mean CPU time across multiple runs, highlighting how increasing the rank/sketch size impacts the overall computational cost for both methods. Left: smile matrix. Right: spiral matrix.

For the most extreme case, occurring at rank $\ell = 125$ in the smile matrix, we observe a $\frac{5.556 \text{ s}}{0.089 \text{ s}} \approx 63$ times efficiency increase when constructing the RPCholesky preconditioner compared to the Nyström preconditioner. The spiral matrix exhibits similar performance gains, with the RPCholesky method achieving approximately $\frac{5.467 \text{ s}}{0.101 \text{ s}} \approx 54$ times faster construction time at the same rank. Given the observed growth pattern, this performance gap would likely widen even further at higher ranks, potentially reaching several orders of magnitude for larger problem sizes. This scaling behavior renders the Nyström preconditioner impractical for large-scale applications, which are common in real-world scenarios where matrices of significantly larger dimensions are often encountered. In contrast, the RPCholesky method maintains CPU times consistently below 0.1 seconds, even at the highest tested rank, suggesting that it's viable for substantially larger problem sizes without demanding unreasonable computational resources. This exceptional scalability, combined with consistent performance across both matrices, establishes RPCholesky as the superior choice for large-scale applications involving Gaussian kernel matrices.

Given the very high preconditioning time of the Nyström method, we investigated whether using preconditioning was even advantageous compared to the unpreconditioned system. We ran the experiment again, comparing both preconditioners' CPU time against the unpreconditioned system, which yielded the following results:

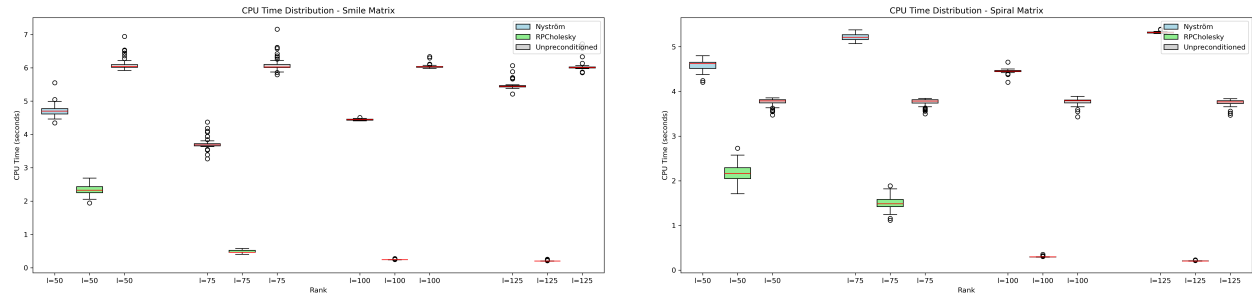


Figure 5: Boxplot distributions of the CPU time required for the preconditioned and unpreconditioned systems to converge. Left boxplots show the Nyström preconditioner, middle boxplots show the RPCholesky preconditioner, and right boxplots show the unpreconditioned system.

We also did the same for the iteration count:

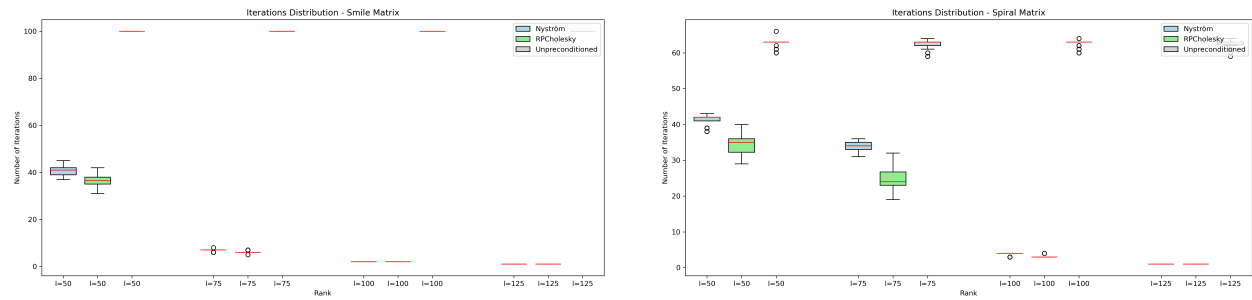


Figure 6: Boxplot distributions of the number of iterations required for the preconditioned and unpreconditioned systems to converge. Left boxplots show the Nyström preconditioner, middle boxplots show the RPCholesky preconditioner, and right boxplots show the unpreconditioned system.

Figure 5 demonstrates that for the smile matrix, while the RPCholesky preconditioner significantly outperforms the Nyström preconditioner, the latter still shows better performance than the unpreconditioned system. However, given the steep increase in CPU time with rank, we anticipate that at rank $\ell = 150$, the unpreconditioned system would already become more efficient than the Nyström preconditioner. For the spiral matrix we also see the RPCholesky preconditioner massively outperforming the unpreconditioned system whereas the Nyström-preconditioner doesn't improve the CPU time at all. Figure 6 reveals that both preconditioners substantially improve the convergence rate, which can be attributed to their effectiveness in reducing the condition number. An additional insight from Figure 6 concerns the Conjugate Gradient method's sensitivity to the right-hand side vector b for different matrices. While the method shows consistent behavior for the smile matrix, it exhibits notable variance in iteration counts for the spiral matrix. Since the CG method is deterministic and only the right-hand side vector b varies between instances, this variance must be attributed to the method's different responses to various b vectors when applied to the spiral matrix.

4 Discussion

The numerical experiments presented in the previous section highlights the value of low-rank approximations in preconditioning the Conjugate Gradient (CG) method for large-scale symmetric positive definite systems. Both the randomized Nyström preconditioner and the randomly pivoted Cholesky preconditioner successfully reduced iteration counts relative to the unpreconditioned system. However, their total computational costs—taking into account both the construction of the preconditioner and the CG phase—varied considerably.

From our experiments with Gaussian kernel matrices of dimension $n = 1000$, specifically Figure 1, we observed that the randomized Nyström method consistently made for a good approximation of A at ranks considerably lower than n . This is reflected by the notable improvements in the convergence rate observed in Figure 6. However, its computational overhead for constructing the preconditioner escalates rapidly with increasing rank according to 5. This high cost comes from 1. forming and multiplying with the sketch matrix Ω , which requires $\mathcal{O}(n^2\ell)$ operations for the matrix-matrix multiplication $A\Omega$ in the case of dense unstructured matrices. In our case, since our test matrices are constructed using an exponential function (see (19)), every entry is non-zero by the properties of the exponential function, resulting in fully dense matrices where this bound applies. 2. performing multiple matrix decompositions including the QR decomposition of Ω requiring $\mathcal{O}(n\ell^2)$ operations, the Cholesky decomposition of $\Omega^T Y_\nu$ requiring $\mathcal{O}(\ell^3)$ operations, and the final SVD of B requiring $\mathcal{O}(n\ell^2)$ operations.[7] The current implementation could probably be improved in several ways however it would still suffer from the inherent computational complexity of operating on dense matrices.

By contrast, RPCholesky stands out for its relatively lower precomputation overhead. Its pivoting strategy incrementally selects columns, thereby effectively capturing dominant directions of A . Across the tested rank values, this approach maintained consistently low overhead times while still providing an as good or even better reductions in iterations compared to the Randomized Nyström preconditioner. In practical terms, such scalability becomes critical for large problems where computational resources—both time and memory—are highly constrained making it preferable over the Randomized Nyström preconditioner.

In regards to the influence of the rank parameter ℓ both methods showed improved conditioning, that is a lower condition number, of the system matrix as ℓ increased, which translated into fewer CG iterations as seen in Figure 1. However in Figure 2, we saw that in the context of overall runtime, a higher ℓ did not always yield faster convergence in CPU time for the randomized Nyström preconditioner. The precomputation overhead grew faster than the iteration savings could compensate. For RPCholesky, by contrast, the total time mostly followed the trend of reduced CG iterations without incurring a substantial additional precomputation cost.

The experiments also highlighted how the distribution of eigenvalues (i.e., the spectrum) of the system matrix interacts with the right-hand side vector b . In particular, in Figure 6 we observe a variation in iteration counts across repeated trials with different right hand for the unpreconditioned spiral matrix. This suggests that CG’s convergence sensitivity depends not only on eigenvalue clustering but also on how b projects onto eigenvector subspaces.

In practice, such sensitivity might be mitigated by better-informed initial guesses, adaptive preconditioners, or more robust pivot rules.

While our experiments provided valuable insights into the behavior of the two preconditioners, several promising directions warrant further investigation. Specifically since we only limited our attention to Gaussian kernel matrices in our experiments, which are examples of dense symmetric positive definite matrices, employing the algorithms on other kinds of matrices would give a better idea of their potential use cases. Real-world applications often produce sparse symmetric matrices or involve additional structure (banded, block, etc.), making the exploration of algorithm performance across these different matrix types particularly relevant. Furthermore we only considered ranks ℓ up to 125 and matrices of size $n = 1000$. In more extreme large-scale settings, the choice of ℓ would show an even greater importance in terms of performance, and the balance between iteration savings and precomputation cost might shift. Future work could explore the performance and trade-offs between precomputation costs and iteration savings in settings where n and ℓ are substantially larger, such as $n \gg 1000$ and $\ell \gg 125$. While we included a small regularization term μI to ensure numerical stability, a deeper study of how μ interacts with the preconditioner construction and convergence could reveal optimal parameter choices. Adaptive or data-driven methods for selecting μ may lead to improved robustness and efficiency gains. Moreover, this study did not explicitly examine the spectrum of the preconditioned system $P^{-1}A$ and how it influences CG convergence. Future work could delve deeper into the spectral properties of each preconditioned system to show why and how it accelerates convergence as discussed in section 2.8. In particular, it could do this by both computing the eigenvalues of the preconditioned system to see how well the eigenvalue clusters and also by tracking the condition number $\kappa(P^{-1}A)$ quantifying exactly how much each preconditioner improves conditioning. Finally, although we used random right-hand side vectors b , a more systematic investigation of how b aligns with the eigenvectors of $P^{-1}A$ may offer further insight into the interplay between the spectrum of $P^{-1}A$ and convergence properties.

5 Conclusion

In this thesis, we investigated two low-rank preconditioning strategies for accelerating the CG method when solving large-scale symmetric positive definite linear systems. The first strategy employs the randomized Nyström approximation, which constructs a low-rank approximation of the system matrix via random sketches. The second strategy, the random pivoted partial Cholesky (RPCholesky) approach, adaptively selects columns of the matrix to incrementally build a low-rank Cholesky factor. Our experiments focused on Gaussian kernel matrices of dimensions up to $n = 1000$, assessing both methods under various rank parameters ℓ . Both preconditioners demonstrated significant reductions in the number of CG iterations compared to the unpreconditioned baseline, confirming that low-rank approximations can effectively improve the conditioning of large symmetric positive definite matrices. However, a surprising result emerged when measuring total CPU time required to reach convergence. While the Nyström method consistently reduced the iteration count as its rank ℓ increased, the cost of constructing the preconditioner became disproportionately expensive,

ultimately not making non-viable. By contrast, the RPCholesky method showed consistently low construction overhead across the tested rank sizes. Consequently, RPCholesky tended to outperform the Nyström approach in terms of total CPU time, especially at higher ranks. While our implementation of the RPCholesky algorithm could be significantly optimized to reduce CPU time, the results demonstrate an important principle: preconditioners vary in their effectiveness across different applications. This underscores the need to evaluate preconditioner selection on a case-by-case basis. In conclusion, our study reveals that while both preconditioners successfully reduce iteration counts, the RPCholesky approach offers a more practical solution for kernel matrices due to its favorable balance between computational overhead and convergence improvement.

List of Figures

1	Boxplot distributions of how choosing the rank/sketch size affects the number of iterations required for the randomized Nyström PCG and RPCholesky PCG to converge. Left: smile matrix. Right: spiral matrix.	21
2	Visualization of how choosing the rank/sketch size affects the elapsed CPU time required for the randomized Nyström PCG and RPCholesky PCG to converge. Left: smile matrix. Right: spiral matrix.	22
3	Boxplot distributions of CPU time spent constructing the Randomized Nyström and RPCholesky preconditioners at different rank/sketch sizes. Left: smile matrix. Right: spiral matrix.	23
4	Average CPU time required to construct the Randomized Nyström and RPCholesky preconditioners at various rank/sketch sizes. Each bar represents the mean CPU time across multiple runs, highlighting how increasing the rank/sketch size impacts the overall computational cost for both methods. Left: smile matrix. Right: spiral matrix.	24
5	Boxplot distributions of the CPU time required for the preconditioned and unpreconditioned systems to converge. Left boxplots show the Nyström preconditioner, middle boxplots show the RPCholesky preconditioner, and right boxplots show the unpreconditioned system.	25
6	Boxplot distributions of the number of iterations required for the preconditioned and unpreconditioned systems to converge. Left boxplots show the Nyström preconditioner, middle boxplots show the RPCholesky preconditioner, and right boxplots show the unpreconditioned system.	25

List of Tables

References

- [1] J. R. Shewchuk, “An introduction to the conjugate gradient method without the agonizing pain,” tech. rep., USA, 1994.
- [2] Wikipedia contributors, “Lu decomposition — Wikipedia, the free encyclopedia.” https://en.wikipedia.org/w/index.php?title=LU_decomposition&oldid=1245959211, 2024. [Online; accessed 23-September-2024].
- [3] Wikipedia contributors, “Cholesky decomposition — Wikipedia, the free encyclopedia.” https://en.wikipedia.org/w/index.php?title=Cholesky_decomposition&oldid=1243010992, 2024. [Online; accessed 23-September-2024].
- [4] O. Besson and F. Vincent, “Properties of the partial Cholesky factorization and application to reduced-rank adaptive beamforming,” *Signal Processing*, vol. 167, pp. 107300–107309, 2020.
- [5] S. Treil, *Linear Algebra Done Wrong*. Department of Mathematics, Brown University, 2004. Available at <https://www.math.brown.edu/streil/papers/LADW/LADW.html>.
- [6] “Iterative method.” <https://www.sciencedirect.com/topics/mathematics/iterative-method>. Accessed: January 6, 2025.
- [7] Z. Frangella, J. A. Tropp, and M. Udell, “Randomized nyström preconditioning,” *SIAM Journal on Matrix Analysis and Applications*, vol. 44, no. 2, pp. 718–752, 2023.
- [8] Wikipedia contributors, “Vieta’s formulas — Wikipedia, the free encyclopedia.” https://en.wikipedia.org/w/index.php?title=Vieta%27s_formulas&oldid=1240960727, 2024. [Online; accessed 28-October-2024].
- [9] M. S. Andersen, “Krylov subspaces and conjugate gradients.” Working draft, 2 2024.
- [10] Z. Frangella, J. A. Tropp, and M. Udell, “Randomized Nyström Preconditioning,” *SIAM Journal on Matrix Analysis and Applications*, vol. 44, no. 2, pp. 718–752, 2023.
- [11] Y. Chen, E. N. Epperly, J. A. Tropp, and R. J. Webber, “Randomly pivoted Cholesky: Practical approximation of a kernel matrix with few entry evaluations,” *arXiv preprint arXiv:2207.06503*, 2023. Version 5.
- [12] J. D. Hunter *et al.*, “matplotlib.pyplot.boxplot.” https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.boxplot.html, 2024, Version 3.9. [Online; accessed 12-December-2024].
- [13] Y. Chen, E. N. Epperly, J. A. Tropp, and R. J. Webber, “Randomly pivoted cholesky: Practical approximation of a kernel matrix with few entry evaluations,” *arXiv preprint arXiv:2207.06503*, 2023.

-
- [14] E. N. Epperly, J. A. Tropp, and R. J. Webber, “Embrace rejection: Kernel matrix approximation by accelerated randomly pivoted Cholesky,” *Manuscript in preparation*, 2024.

6 Appendix

7 Code appendix

7.1 Code Listing

Listing 1: Test Matrices

```

1  #!/usr/bin/env python3
2
3  import numpy as np
4  from HelperFunctions.matrix import PSDMatrix, KernelMatrix
5  import itertools
6  import scipy.io
7  import re
8  from warnings import warn
9  import os
10
11 def smile(N, bandwidth = 2.0, **kwargs):
12     small = int(np.ceil(N ** (1.0/2)))
13     eye_points = kwargs["eye_points"] if ("eye_points" in kwargs) else small
14     mouth_points = kwargs["mouth_points"] if ("mouth_points" in kwargs) else int(np.ceil(N/10.0))
15     face_points = N - 2 * eye_points - mouth_points
16
17     X = np.zeros((N, 2))
18     idx = 0
19
20     # Eyes
21     for x_shift in [-4.0, 4.0]:
22         for i in range(eye_points):
23             while True:
24                 x = 2 * np.random.rand() - 1
25                 y = 2 * np.random.rand() - 1
26                 if x**2 + y**2 <= 1.0:
27                     X[idx, 0] = x + x_shift
28                     X[idx, 1] = y + 4.0
29                     idx += 1
30                     break
31
32     # Mouth
33     for x in list(np.linspace(-5.0, 5.0, mouth_points)):
34         X[idx, 0] = x
35         X[idx, 1] = x**2 / 16.0 - 5.0
36         idx += 1
37
38     # Face
39     for theta in list(np.linspace(0, 2*np.pi, face_points)):
40         X[idx, 0] = 10.0 * np.cos(theta)
41         X[idx, 1] = 10.0 * np.sin(theta)
42         idx += 1
43
44     if "dimension" in kwargs:
45         d = kwargs["dimension"]
46         Q, _ = np.linalg.qr(np.random.randn(d, 2))
47         Q = Q.T
48         X = X @ Q
49
50     return KernelMatrix(X, bandwidth = bandwidth, **kwargs)
51
52 def expspiral(N, rate=1e-3, rotate_rate=5e-3, power = 1.5, bandwidth = 0.02):
53     X = np.zeros((N, 2))
54     t = np.array(range(N))
55     X[:,0] = np.exp(-rate * N * (t/N) ** power) * np.cos(rotate_rate * t)
56     X[:,1] = np.exp(-rate * N * (t/N) ** power) * np.sin(rotate_rate * t)
57     return KernelMatrix(X, bandwidth = bandwidth)
58
59 def robspiral(N):
60     times = np.linspace(0, 2, N)
61     times = times ** 6
62     times = times[::-1]
63     x = np.exp(.2 * times) * np.cos(times)
64     y = np.exp(.2 * times) * np.sin(times)
65     X = np.column_stack((x,y))
66     bandwidth = 1000
67     return KernelMatrix(X, bandwidth = bandwidth)
68
69 def powerspiral(N, max_radius = 1.0, angle = 0.2, decay_power = 1, bandwidth = 1e-3):
70     X = np.zeros((N, 2))
71     t = np.array(range(1,N+1), dtype=float)
72     radii = max_radius * t ** (-decay_power)
73     angles = angle * t
74     X[:,0] = radii * np.cos(angles)
75     X[:,1] = radii * np.sin(angles)

```

```

76 # np.random.shuffle(X)
77 return KernelMatrix(X, bandwidth = bandwidth)
78
79 def outliers(N, num_outliers = 50):
80     X = 0.5*np.random.randn(N, 20)/np.sqrt(20.0)
81     X[np.random.choice(range(N), size = num_outliers, replace = False),:] += 100.0 * np.random.randn(num_outliers, 20)
82     return KernelMatrix(X)
83
84 def random_kernel_matrix(N, d = 10, **kwargs):
85     X = np.random.randn(N, d)
86     return KernelMatrix(X, **kwargs)
87
88 def kernel_from_data(data, **kwargs):
89     mean = np.mean(data, axis=0)
90     stddev = 0.0
91     for i in range(data.shape[0]):
92         stddev += np.linalg.norm(data[i,:] - mean)**2
93     return KernelMatrix(data, bandwidth = np.sqrt(stddev / data.shape[0]), **kwargs)
94
95 def find_mat_files_and_prefixes(folder_path):
96     mat_files = []
97     mat_prefixes = []
98     pattern = re.compile(r'(.*)\.mat$')
99     for root, dirs, files in os.walk(folder_path):
100         for file in files:
101             match = pattern.match(file)
102             if match:
103                 mat_files.append(os.path.join(root, file))
104                 mat_prefixes.append(match.group(1))
105     return mat_files, mat_prefixes
106
107 def gallery(N, datafolder = None, min_N=None):
108     if min_N is None:
109         min_N = 0
110
111     yield "Smile (high)", smile(N, bandwidth=0.2, extra_stability=True)
112     yield "Smile (medium)", smile(N, bandwidth=0.15, extra_stability=True)
113     yield "Smile (low)", smile(N, bandwidth=0.1, extra_stability=True)
114     yield "Exponential Spiral (high)", expspiral(N,bandwidth=0.02)
115     yield "Exponential Spiral (medium)", expspiral(N,bandwidth=0.015)
116     yield "Exponential Spiral (low)", expspiral(N,bandwidth=0.01)
117     yield "Outliers (50)", outliers(N)
118     yield "Outliers (500)", outliers(N, num_outliers=500)
119     yield "Outliers (5000)", outliers(N, num_outliers=5000)
120
121     for d in [2,10,100,1000]:
122         for bandwidth, bandname in [(2*np.sqrt(d),"high"),(np.sqrt(d),"medium"),(np.sqrt(d)/2,"low")]:
123             for kerneltype in ["laplace", "matern", "gaussian"]:
124                 yield f"Random ({d},{bandname},{kerneltype})", random_kernel_matrix(N, d=d, bandwidth=(bandwidth/10 if kerneltype=="gaussian" else
125                                                             bandwidth), kernel=kerneltype, nu=1.5)
126
127     if datafolder is None:
128         return
129
130     files, names = find_mat_files_and_prefixes(datafolder)
131     for name, myfile in zip(names, files):
132         data = scipy.io.loadmat(myfile)
133         if "Xtr" in data:
134             X = data["Xtr"]
135         elif "A" in data:
136             X = data["A"]
137         else:
138             print(f"Dataset {name} did not contain a variable named 'Xtr' or 'A'. Skipping")
139             continue
140         if type(X) != np.ndarray:
141             X = X.toarray()
142         X = X.astype(float)
143
144         if X.shape[0] < min_N:
145             print(f"Dataset {name} of size {X.shape[0]} is too small. Skipping")
146             continue
147
148         if X.shape[0] > N:
149             X = X[np.random.choice(X.shape[0], N, replace=False),:]
150
151         for kerneltype in ["laplace", "matern", "gaussian"]:
152             A = KernelMatrix(X, kerneltype, bandwidth = "approx_median", nu=1.5)
153             yield f"{name} ({kerneltype})", A

```

Listing 2: Kernel Matrix

```

1 #!/usr/bin/env python3
2
3 import numpy as np
4 from scipy.special import gamma, kv

```

```

5 from sklearn.metrics.pairwise import euclidean_distances, manhattan_distances
6
7 def LaplaceKernel(x,y,bandwidth=1.0,**kwargs):
8     # for single x,y in R^d
9     return np.exp(-np.linalg.norm(x-y, ord=1) / bandwidth )
10
11 def LaplaceKernel_vec(vec_x,vec_y,bandwidth=1.0,**kwargs):
12     # for vec_x, vec_y in R^{n*d}, return n values
13     dsts = np.linalg.norm(vec_x-vec_y, ord = 1, axis = -1)
14     return np.exp(-dsts/bandwidth)
15
16 def LaplaceKernel_mtx(xx,yy,bandwidth=1.0,**kwargs):
17     # xx in R^{n*d} and yy in R^{ny*d} are both collection of points (two axis)
18     # return nx*ny values
19     # dsts = np.linalg.norm(xx[:, None, :] - yy[None, :, :], axis=-1)
20     dsts = manhattan_distances(xx,yy) # faster
21     return np.exp(-dsts/bandwidth)
22
23 def GaussianKernel(x,y,bandwidth=1.0,**kwargs):
24     # for single x,y in R^d
25     return np.exp(-0.5*np.linalg.norm(x-y)**2/bandwidth**2)
26
27 def GaussianKernel_vec(vec_x,vec_y, bandwidth=1.0,**kwargs):
28     # for vec_x, vec_y in R^{n*d}, return n values
29     dsts = np.linalg.norm(vec_x-vec_y, axis = -1)**2
30     return np.exp(-0.5*dsts**2/bandwidth**2)
31
32 def GaussianKernel_mtx(xx,yy,bandwidth=1.0,extra_stability=False):
33     # xx in R^{n*d} and yy in R^{ny*d} are both collection of points (two axis)
34     # return nx*ny values
35     if extra_stability:
36         dsts = np.linalg.norm(xx[:, None, :] - yy[None, :, :], axis=-1)
37     else:
38         dsts = euclidean_distances(xx,yy) # faster
39     return np.exp(-0.5*dsts**2/bandwidth**2)
40
41 def MaternKernel(x,y,bandwidth=1.0, nu=0.5,**kwargs):
42     # for single x,y in R^d
43     d = np.linalg.norm(x-y) / bandwidth
44     if nu == 0.5:
45         return np.exp(-d)
46     elif nu == 1.5:
47         sqrt3 = np.sqrt(3.0)
48         return (1 + sqrt3*d) * np.exp(-sqrt3*d)
49     elif nu == 2.5:
50         sqrt5 = np.sqrt(5.0)
51         return (1 + sqrt5*d + 5.0/3.0*d*d) * np.exp(-sqrt5*d)
52     else:
53         sqrt2nu = np.sqrt(2.0*nu)
54         return 1.0/(gamma(nu) * 2.0**(nu-1)) * (sqrt2nu * d) ** nu * kv(nu, sqrt2nu * d)
55
56 def MaternKernel_vec(vec_x,vec_y,bandwidth=1.0, nu=0.5,**kwargs):
57     # for vec_x, vec_y in R^{n*d}, return n values
58     d = np.linalg.norm(vec_x-vec_y, axis = -1)/bandwidth
59     if nu == 0.5:
60         return np.exp(-d)
61     elif nu == 1.5:
62         sqrt3 = np.sqrt(3.0)
63         return (1 + sqrt3*d) * np.exp(-sqrt3*d)
64     elif nu == 2.5:
65         sqrt5 = np.sqrt(5.0)
66         return (1 + sqrt5*d + 5.0/3.0*d*d) * np.exp(-sqrt5*d)
67     else:
68         sqrt2nu = np.sqrt(2.0*nu)
69         return 1.0/(gamma(nu) * 2.0**(nu-1)) * (sqrt2nu * d) ** nu * kv(nu, sqrt2nu * d)
70
71 def MaternKernel_mtx(xx,yy,bandwidth,nu,extra_stability=False):
72     # xx in R^{n*d} and yy in R^{ny*d} are both collection of points (two axis)
73     # return nx*ny values
74     if extra_stability:
75         d = np.linalg.norm(xx[:, None, :] - yy[None, :, :], axis=-1)
76     else:
77         d = euclidean_distances(xx,yy)/bandwidth # faster
78     if nu == 0.5:
79         return np.exp(-d)
80     elif nu == 1.5:
81         sqrt3 = np.sqrt(3.0)
82         return (1 + sqrt3*d) * np.exp(-sqrt3*d)
83     elif nu == 2.5:
84         sqrt5 = np.sqrt(5.0)
85         return (1 + sqrt5*d + 5.0/3.0*d*d) * np.exp(-sqrt5*d)
86     else:
87         sqrt2nu = np.sqrt(2.0*nu)
88         return 1.0/(gamma(nu) * 2.0**(nu-1)) * (sqrt2nu * d) ** nu * kv(nu, sqrt2nu * d)

```

Listing 3: First Experiment: Iterations

```

1 import numpy as np
2 from typing import Dict, List, Any
3 import matplotlib.pyplot as plt
4 from PCG import pcg_solver
5 from RPCholesky import rpcholesky
6 from HelperFunctions.gallery import smile, robspiral
7 from tqdm import tqdm
8
9 def run_experiment(matrix_name: str, A: Any, ranks: List[int],
10                   num_instances: int, threshold: float, mu: float) -> Dict:
11     """
12     Run experiment comparing Nyström PCG with RPCholesky PCG
13     """
14     n = A.shape[0]
15     results = {
16         'nystrom': {r: {'iters': []} for r in ranks},
17         'rpcholesky': {r: {'iters': []} for r in ranks}
18     }
19
20     # Set random seed for reproducibility
21     np.random.seed(101)
22
23     for rank in ranks:
24         print(f"Processing rank {rank} for {matrix_name}")
25
26         for instance in tqdm(range(num_instances), desc=f"Rank {rank} Instances", position=1, leave=False):
27             # Generate random right-hand side vector
28             b = np.random.randn(n)
29             x0 = np.zeros(n)
30
31             # Run Nyström PCG
32             x_nys, nys_iters, nys_error = pcg_solver(A, b, x0, mu, rank, 'nystrom', threshold)
33
34             # Run RPCholesky PCG
35             x_rp, rp_iters, rp_error = pcg_solver(A, b, x0, mu, rank, 'rpcholesky', threshold)
36
37             # Store results
38             results['nystrom'][rank]['iters'].append(nys_iters)
39             results['rpcholesky'][rank]['iters'].append(rp_iters)
40
41     return results
42
43 def plot_boxplot(results: Dict, ranks: List[int], title: str, output_dir: str = 'plots'):
44     """
45     Create boxplot comparing the iteration distributions and save as PNG
46     """
47     plt.figure(figsize=(10, 6), dpi=300)
48
49     # Prepare data for boxplot
50     iters_data_nystrom = [results['nystrom'][rank]['iters'] for rank in ranks]
51     iters_data_rpcholesky = [results['rpcholesky'][rank]['iters'] for rank in ranks]
52
53     # Position for boxes
54     positions_nystrom = np.arange(len(ranks)) * 2.5
55     positions_rpcholesky = positions_nystrom + 0.8
56
57     # Plot iterations boxplot
58     plt.boxplot(iters_data_nystrom, positions=positions_nystrom, widths=0.6,
59                 patch_artist=True, boxprops=dict(facecolor='lightblue', color='black'),
60                 medianprops=dict(color='red'), labels=[f'l={r}' for r in ranks])
61     plt.boxplot(iters_data_rpcholesky, positions=positions_rpcholesky, widths=0.6,
62                 patch_artist=True, boxprops=dict(facecolor='lightgreen', color='black'),
63                 medianprops=dict(color='red'), labels=[f'l={r}' for r in ranks])
64
65     # Customize plot
66     plt.xlabel('Rank')
67     plt.ylabel('Number of Iterations')
68     plt.title(f'Iterations Distribution — {title}')
69
70     # Add legend
71     from matplotlib.patches import Patch
72     legend_elements = [
73         Patch(facecolor='lightblue', edgecolor='black', label='Nyström'),
74         Patch(facecolor='lightgreen', edgecolor='black', label='RPCholesky')
75     ]
76     plt.legend(handles=legend_elements, loc='upper right')
77
78     plt.tight_layout()
79
80     # Create output directory if it doesn't exist
81     import os
82     os.makedirs(output_dir, exist_ok=True)
83
84     # Save plot as PNG with high DPI

```



```

85     filename = os.path.join(output_dir, f'{title.lower().replace(" ", "_")}_boxplot.png')
86     plt.savefig(filename, dpi=300, bbox_inches='tight')
87     plt.close()
88     print(f"Plot saved as {filename}")
89
90 def print_summary_statistics(matrix_name: str, results: Dict, ranks: List[int]):
91     """Print summary statistics for a single experiment"""
92     print(f"\n{matrix_name} Matrix:")
93     for method in ['nystrom', 'rpcholesky']:
94         print(f"\n{method.capitalize()} method:")
95         for rank in ranks:
96             avg_iters = np.mean(results[method][rank]['iters'])
97             print(f"Rank {rank}: Avg iterations = {avg_iters:.1f}")
98
99 def plot_histogram(results: Dict, ranks: List[int], title: str, output_dir: str = 'plots'):
100     """
101     Create histogram comparing average iterations between methods and save as PNG
102     """
103     plt.figure(figsize=(10, 6), dpi=300)
104
105     # Calculate averages for each rank
106     nystrom_avgs = [np.mean(results['nystrom'][rank]['iters']) for rank in ranks]
107     rpcholesky_avgs = [np.mean(results['rpcholesky'][rank]['iters']) for rank in ranks]
108
109     # Set up bar positions
110     x = np.arange(len(ranks))
111     width = 0.35 # Width of the bars
112
113     # Create bars
114     plt.bar(x - width/2, nystrom_avgs, width, label='Nyström', color='lightblue', edgecolor='black')
115     plt.bar(x + width/2, rpcholesky_avgs, width, label='RPCholesky', color='lightgreen', edgecolor='black')
116
117     # Customize plot
118     plt.xlabel('Rank')
119     plt.ylabel('Average Number of Iterations')
120     plt.title(f'Average Iterations Comparison — {title}')
121     plt.xticks(x, [f'l={r}' for r in ranks])
122     plt.legend()
123
124     # Add value labels on top of bars
125     for i, v in enumerate(nystrom_avgs):
126         plt.text(i - width/2, v + 1, f'{v:.1f}', ha='center', va='bottom')
127     for i, v in enumerate(rpcholesky_avgs):
128         plt.text(i + width/2, v + 1, f'{v:.1f}', ha='center', va='bottom')
129
130     plt.tight_layout()
131
132     # Create output directory if it doesn't exist
133     import os
134     os.makedirs(output_dir, exist_ok=True)
135
136     # Save plot as PNG
137     filename = os.path.join(output_dir, f'{title.lower().replace(" ", "_")}_histogram.png')
138     plt.savefig(filename, dpi=300, bbox_inches='tight')
139     plt.close()
140     print(f"Histogram saved as {filename}")
141
142 def run_smile_experiment():
143     """Run experiment for Smile matrix"""
144     # Parameters
145     n = 1000
146     ranks = [50, 75, 100, 125]
147     num_instances = 50
148     threshold = 1e-4
149     mu = 1e-3
150
151     print("Generating Smile matrix...")
152     A_smile = smile(n, bandwidth=2.0)
153
154     print("\nRunning experiments for Smile matrix...")
155     results_smile = run_experiment('Smile', A_smile, ranks, num_instances, threshold, mu)
156
157     print("\nPlotting results...")
158     plot_boxplot(results_smile, ranks, 'Smile Matrix') # Add boxplot
159     plot_histogram(results_smile, ranks, 'Smile Matrix') # Add histogram
160
161     print("\nSummary Statistics:")
162     print_summary_statistics("Smile", results_smile, ranks)
163
164     return results_smile
165
166 def run_spiral_experiment():
167     """Run experiment for Spiral matrix"""
168     # Parameters
169     n = 1000
170     ranks = [50, 75, 100, 125]

```

```

171 num_instances = 50
172 threshold = 1e-4
173 mu = 1e-2
174
175 print("Generating Spiral matrix...")
176 A_spiral = robspiral(n)
177
178 print("\nRunning experiments for Spiral matrix...")
179 results_spiral = run_experiment('Spiral', A_spiral, ranks, num_instances, threshold, mu)
180
181 print("\nPlotting results...")
182 plot_boxplot(results_spiral, ranks, 'Spiral Matrix')
183 plot_histogram(results_spiral, ranks, 'Spiral Matrix') # Add histogram
184
185 print("\nSummary Statistics:")
186 print_summary_statistics("Spiral", results_spiral, ranks)
187
188 return results_spiral
189
190 if __name__ == "__main__":
191     import sys
192
193     if len(sys.argv) != 2 or sys.argv[1] not in ['smile', 'spiral', 'both']:
194         print("Usage: python script.py [smile|spiral|both]")
195         sys.exit(1)
196
197     if sys.argv[1] == 'smile' or sys.argv[1] == 'both':
198         results_smile = run_smile_experiment()
199
200     if sys.argv[1] == 'spiral' or sys.argv[1] == 'both':
201         results_spiral = run_spiral_experiment()

```

Listing 4: Preconditioned Conjugate Gradient Method

```

1 import numpy as np
2 from typing import Tuple, Any
3 from RandomizedNystrom import randomized_nystrom_approximation
4 from RPCholesky import rpcholesky
5
6 def pcg_solver(A: Any, b: np.ndarray, x0: np.ndarray, mu: float,
7               sketch_size: int, precondition_type: str,
8               eta: float = 1e-6, max_iter: int = 5000) -> Tuple[np.ndarray, int, float]:
9     """
10     Implements PCG with either Nystrom or RPCholesky preconditioner for solving regularized linear systems
11
12     Args:
13         A: PSD matrix
14         b: right-hand side vector
15         x0: initial guess
16         mu: regularization parameter
17         sketch_size: size of the sketch (l)
18         precondition_type: type of preconditioner ('nystrom' or 'rpcholesky')
19         eta: solution tolerance
20         max_iter: maximum number of iterations
21
22     Returns:
23         x: Approximate solution to the regularized system
24         num_iters: Number of iterations performed
25         rel_error: Final relative error
26     """
27     # Step 1: Compute approximation based on preconditioner type
28     if precondition_type.lower() == 'nystrom':
29         U, Lambda = randomized_nystrom_approximation(A, sketch_size)
30     elif precondition_type.lower() == 'rpcholesky':
31         nystrom_approximation = rpcholesky(A, sketch_size)
32         F = nystrom_approximation.get_left_factor()
33         # Convert FF^T to USU^T form using SVD
34         U, S, _ = np.linalg.svd(F, full_matrices=False)
35         Lambda = S**2 # Square singular values to get eigenvalues
36     else:
37         raise ValueError("precond_type must be either 'nystrom' or 'rpcholesky'")
38
39     def apply_preconditioner(r: np.ndarray) -> np.ndarray:
40         """
41         Applies the preconditioner P^{-1} to vector r using equation (15)
42         """
43         # Get smallest eigenvalue
44         lambda_l = np.min(Lambda)
45
46         # First term: (I^T \hat{A} + \hat{I} \hat{I}) (I + \hat{I} \hat{I})^{-1} U^T r
47         Ut_r = U.T @ r
48         Lambda_inv = 1.0 / (Lambda + mu)
49         first_term = (lambda_l + mu) * (U @ (Lambda_inv * Ut_r))
50
51         # Second term: (I - UU^T)r

```

```

52     second_term = r - U @ (U.T @ r)
53
54     return first_term + second_term
55
56 # Initialize
57 r0 = b - ((A + mu * np.eye(A.shape[0])) @ x0)
58 z0 = apply_preconditioner(r0)
59 p = z0
60 x = x0
61
62 # Main PCG loop
63 num_iters = 0
64 for i in range(max_iter):
65     num_iters = i + 1
66
67     v = (A + mu * np.eye(A.shape[0])) @ p
68     alpha = (r0.T @ z0) / (p.T @ v)
69     x = x + alpha * p
70     r = r0 - alpha * v
71     z = apply_preconditioner(r)
72     beta = (r.T @ z) / (r0.T @ z0)
73
74     r0 = r
75     p = z + beta * p
76     z0 = z
77
78 # Check convergence
79 rel_error = np.linalg.norm(r0) / np.linalg.norm(b)
80 if rel_error <= eta:
81     break
82
83 return x, num_iters, rel_error

```

Listing 5: Randomized Nyström Preconditioner

```

1  import numpy as np
2  from typing import Tuple, Any
3  from scipy.linalg import qr, cholesky, svd
4
5  def randomized_nystrom_approximation(A: Any, sketch_size: int) -> Tuple[np.ndarray, np.ndarray]:
6      """
7      Implements Algorithm 1: Randomized Nyström Approximation
8
9      Args:
10         A: Symmetric PSD matrix A ∈ R^{n×n}
11         sketch_size: rank l for approximation
12
13     Returns:
14         U: Orthogonal factor of the Nyström approximation
15         Lambda: Diagonal matrix (eigenvalues) such that A_nys = UΛU^T
16     """
17     n = A.shape[0]
18
19     # Step 1: Generate Gaussian test matrix
20     Omega = np.random.randn(n, sketch_size)
21
22     # Step 2: Thin QR decomposition
23     Omega = np.linalg.qr(Omega, mode='reduced')[0]
24
25     # Step 3: Multiply by A to get â matvecs
26     Y = A @ Omega
27
28     # Step 4: Compute stability shift
29     nu = np.finfo(float).eps * np.linalg.norm(Y, 'fro')
30
31     # Step 5: Apply stability shift
32     Y_nu = Y + nu * Omega
33
34     # Step 6: Cholesky decomposition of ÎŜ^T Y_Î
35     C = np.linalg.cholesky(Omega.T @ Y_nu)
36
37     # Step 7: Compute B = Y_ÎÎ^* / C
38     B = Y_nu @ np.linalg.inv(C.T)
39
40     # Step 8: Thin SVD
41     U, Sigma, _ = np.linalg.svd(B, full_matrices=False)
42
43     # Step 9: Remove shift and compute eigenvalues
44     # Ît' = max{0, ÎtÎ - Î^*Î}
45     Lambda = np.maximum(0, Sigma**2 - nu)
46
47     return U, Lambda

```

Listing 6: RPCholesky Preconditioner

```

1  #!/usr/bin/env python3
2
3  import numpy as np
4  import scipy as sp
5  from HelperFunctions.lra import PSDLowRank
6  from warnings import warn
7  from time import time
8  from HelperFunctions.matrix import AbstractPSDMatrix, PSDMatrix
9
10 greedy_lapack = sp.linalg.get_lapack_funcs('pstrf', dtype=np.float64)
11
12 def cholesky_helper(A, k, alg, stoptol = 0):
13     if not isinstance(A, AbstractPSDMatrix):
14         A = PSDMatrix(A)
15
16     n = A.shape[0]
17     diags = A.diag()
18     orig_trace = sum(diags)
19     if stoptol is None:
20         stoptol = 0
21
22     # row ordering, is much faster for large scale problems
23     G = np.zeros((k,n))
24     rows = np.zeros((k,n))
25     rng = np.random.default_rng()
26
27     arr_idx = []
28
29     for i in range(k):
30         if alg == 'rp':
31             idx = rng.choice(range(n), p = diags / sum(diags))
32         elif alg == 'rgreedy':
33             idx = rng.choice(np.where(diags == np.max(diags))[0])
34         elif alg == "greedy":
35             idx = np.argmax(diags)
36         else:
37             raise RuntimeError("Algorithm '{}' not recognized".format(alg))
38
39         arr_idx.append(idx)
40         rows[i,:] = A[idx,:].T
41         G[i,:] = (rows[i,:] - G[:i,idx].T @ G[:i,:]) / np.sqrt(diags[idx])
42         diags -= G[i,:]**2
43         diags = diags.clip(min = 0)
44
45         if stoptol > 0 and sum(diags) <= stoptol * orig_trace:
46             G = G[:i,:]
47             rows = rows[:i,:]
48             break
49
50     return PSDLowRank(G, idx = arr_idx, rows = rows)
51
52 def _greedy_cholesky(A, rtol = None, atol = None):
53     trace = np.trace(A)
54     L, piv, rank, info = greedy_lapack(A, lower=True)
55     L = np.tril(L)
56     if not (rtol is None) and not (atol is None):
57         if atol is None:
58             atol = 0
59         if rtol is None:
60             rtol = 0
61         trailing_sums = trace - np.cumsum(np.linalg.norm(L, axis=0)**2)
62         rank = np.argmax(trailing_sums < (atol + rtol * trace)) + 1
63     return L, piv, rank
64
65 def block_cholesky_helper(A, k, b, alg, stoptol = 1e-14, strategy = "regularize", rbrp_atol = 0.0, rbrp_rtol = "1/b"):
66     if not isinstance(A, AbstractPSDMatrix):
67         A = PSDMatrix(A)
68
69     diags = A.diag()
70     n = A.shape[0]
71     orig_trace = sum(diags)
72     scale = 2*max(diags)
73     if stoptol is None:
74         stoptol = 1e-14
75     if "1/b" == rbrp_rtol:
76         rbrp_rtol = 1.0/b
77
78     # row ordering
79     G = np.zeros((k,n))
80     rows = np.zeros((k,n))
81
82     rng = np.random.default_rng()
83
84     arr_idx = []

```

```

85
86
87 counter = 0
88 while counter < k:
89     block_size = min(k-counter, b)
90
91     if alg == 'rp':
92         idx = rng.choice(range(n), size = 2*block_size, p = diags / sum(diags), replace = True)
93         idx = np.unique(idx)[:block_size]
94         block_size = len(idx)
95     elif alg == 'greedy':
96         idx = np.argmax(diags, -block_size)[-block_size:]
97     else:
98         raise RuntimeError("Algorithm '{}' not recognized".format(alg))
99
100 if "regularize" == strategy or "regularized" == strategy:
101     arr_idx.extend(idx)
102     rows[counter:counter+block_size,:] = A[idx,:]
103     G[counter:counter+block_size,:] = rows[counter:counter+block_size,:] - G[0:counter,idx].T @ G[0:counter,:]
104     C = G[counter:counter+block_size,idx]
105
106     try:
107         L = np.linalg.cholesky(C + np.finfo(float).eps*b*scale*np.identity(block_size))
108         G[counter:counter+block_size,:] = np.linalg.solve(L, G[counter:counter+block_size,:])
109
110     except np.linalg.LinAlgError:
111         warn("Cholesky failed in block partial Cholesky. Falling back to eigendecomposition")
112         evals, evecs = np.linalg.eigh(C)
113         evals[evals > 0] = evals[evals > 0] ** (-0.5)
114         evals[evals < 0] = 0
115         G[counter:counter+block_size,:] = evals[:,np.newaxis] * (evecs.T @ G[counter:counter+block_size,:])
116
117 elif "rbpr" == strategy or "pivoting" == strategy or "pivoted" == strategy:
118     H = A[idx,idx] - G[0:counter,idx].T @ G[0:counter,idx]
119     L, piv, rank = _greedy_cholesky(H, rtol = rbrp_rtol, atol = rbrp_atol)
120     idx = idx[piv[0:rank]-1]
121     arr_idx.extend(idx)
122     rows[counter:counter+len(idx),:] = A[idx,:]
123     G[counter:counter+len(idx),:] = rows[counter:counter+len(idx),:] - G[0:counter,idx].T @ G[0:counter,:]
124     G[counter:counter+len(idx),:] = np.linalg.solve(L[0:rank,0:rank], G[counter:counter+len(idx),:])
125
126 else:
127     raise ValueError("'{}' is not a valid strategy for block RPCholesky".format(strategy))
128
129 diags -= np.sum(G[counter:counter+len(idx),:]**2, axis=0)
130 diags = diags.clip(min = 0)
131
132 counter += len(idx)
133
134 if stoptol > 0 and sum(diags) <= stoptol * orig_trace:
135     G = G[:counter,:]
136     rows = rows[:counter,:]
137     break
138
139 return PSDLowRank(G, idx = arr_idx, rows = rows)
140
141 def rejection_cholesky(H):
142     b = H.shape[0]
143     if H.shape[0] != H.shape[1]:
144         raise RuntimeError("rejection_cholesky requires a square matrix")
145     if np.trace(H) <= 0:
146         raise RuntimeError("rejection_cholesky requires a strictly positive trace")
147     u = np.array([H[j,j] for j in range(b)])
148
149     idx = []
150     L = np.zeros((b,b))
151     for j in range(b):
152         if np.random.rand() * u[j] < H[j,j]:
153             idx.append(j)
154             L[j,:,j] = H[j,:,j] / np.sqrt(H[j,j])
155             H[(j+1):,(j+1):] -= np.outer(L[(j+1):,j], L[(j+1):,j])
156     idx = np.array(idx)
157     L = L[np.ix_(idx,idx)]
158     return L, idx
159
160 def accelerated_rpcholesky(A, k, b = "auto", stoptol = 1e-13, verbose=False):
161     if not isinstance(A, AbstractPSDMatrix):
162         A = PSDMatrix(A)
163
164     diags = A.diag()
165     n = A.shape[0]
166     orig_trace = sum(diags)
167     if stoptol is None:
168         stoptol = 1e-13
169
170     if "auto" == b:
171         b = int(np.ceil(k / 10))

```

```

171     auto_b = True
172 else:
173     auto_b = False
174
175 # row ordering
176 G = np.zeros((k,n))
177 rows = np.zeros((k,n))
178
179 rng = np.random.default_rng()
180 arr_idx = np.zeros(k, dtype=int)
181
182 counter = 0
183 while counter < k:
184     idx = rng.choice(range(n), size = b, p = diags / sum(diags), replace=True)
185
186     if auto_b:
187         start = time()
188
189     H = A[idx, idx] - G[0:counter,idx].T @ G[0:counter,idx]
190     L, accepted = rejection_cholesky(H)
191     num_sel = len(accepted)
192
193     if num_sel > k - counter:
194         num_sel = k - counter
195         accepted = accepted[:num_sel]
196         L = L[:num_sel,:num_sel]
197
198     idx = idx[accepted]
199
200     if auto_b:
201         rejection_time = time() - start
202         start = time()
203
204     arr_idx[counter:counter+num_sel] = idx
205     rows[counter:counter+num_sel,:] = A[idx,:]
206     G[counter:counter+num_sel,:] = rows[counter:counter+num_sel,:] - G[0:counter,idx].T @ G[0:counter,:]
207     G[counter:counter+num_sel,:] = np.linalg.solve(L, G[counter:counter+num_sel,:])
208     diags -= np.sum(G[counter:counter+num_sel,:]**2, axis=0)
209     diags = diags.clip(min = 0)
210
211     if auto_b:
212         process_time = time() - start
213
214         # Assuming rejection_time ~ A b^2 and process_time ~ C b
215         # then obtaining rejection_time = process_time / 4 entails
216         # b = C / 4A = (process_time / b) / 4 (rejection_time / b^2)
217         # b = b * process_time / (4 * rejection_time)
218         target = int(np.ceil(b * process_time / (4 * rejection_time)))
219         b = max([min([target, int(np.ceil(1.5*b)), int(np.ceil(k/3))],
220                     int(np.ceil(b/3)), 10])
221
222     counter += num_sel
223
224     if stoptol > 0 and sum(diags) <= stoptol * orig_trace:
225         G = G[:counter,:]
226         rows = rows[:counter,:]
227         break
228
229     if verbose:
230         print("Accepted {} / {}".format(num_sel, b))
231
232 return PSDLowRank(G, idx = arr_idx, rows = rows)
233
234 def rpcholesky(A, k, b = None, accelerated = True, **kwargs):
235     if b is None:
236         if accelerated:
237             return accelerated_rpcholesky(A, k, **kwargs)
238         else:
239             return cholesky_helper(A, k, 'rp', **kwargs)
240     elif accelerated:
241         return accelerated_rpcholesky(A, k, b, **kwargs)
242     else:
243         return block_cholesky_helper(A, k, b, 'rp', **kwargs)
244
245 def greedy(A, k, randomized_tiebreaking = False, b = 1, **kwargs):
246     if b == 1:
247         return cholesky_helper(A, k, 'rgreedy' if randomized_tiebreaking else 'greedy', **kwargs)
248     else:
249         if randomized_tiebreaking:
250             warn("Randomized tiebreaking not implemented for block greedy method")
251         return block_cholesky_helper(A, k, b, 'greedy', **kwargs)
252
253 def simple_rpcholesky(A, k, **kwargs):
254     return rpcholesky(A, k, b = None, accelerated = False, **kwargs)
255
256 def block_rpcholesky(A, k, b = 100, **kwargs):

```

```

257     return block_cholesky_helper(A, k, b, 'rp', **kwargs)
258
259 def block_greedy(A, k, b = 100, stoptol = None):
260     return block_cholesky_helper(A, k, b, 'greedy', **kwargs)

```

Listing 7: Second Experiment: CPU-time

```

1  import numpy as np
2  from typing import Dict, List, Any
3  import matplotlib.pyplot as plt
4  from PCG import pcg_solver
5  from RPCholesky import rpcholesky
6  from RandomizedNystrom import randomized_nystrom_approximation
7  from HelperFunctions.gallery import smile, robspiral
8  from tqdm import tqdm
9  import time
10
11 def run_experiment(matrix_name: str, A: Any, ranks: List[int],
12                   num_instances: int, threshold: float, mu: float) -> Dict:
13     """
14     Run experiment comparing Nyström PCG with RPCholesky PCG, measuring total CPU time
15     """
16     n = A.shape[0]
17     results = {
18         'nystrom': {r: {'times': []} for r in ranks},
19         'rpcholesky': {r: {'times': []} for r in ranks}
20     }
21
22     # Set random seed for reproducibility
23     np.random.seed(101)
24
25     for rank in ranks:
26         print(f"Processing rank {rank} for {matrix_name}")
27
28         for instance in tqdm(range(num_instances), desc=f"Rank {rank} Instances", position=1, leave=False):
29             # Generate random right-hand side vector
30             b = np.random.randn(n)
31             x0 = np.zeros(n)
32
33             # Run Nyström PCG with timing
34             start_time = time.process_time()
35             x_nys, _, _ = pcg_solver(A, b, x0, mu, rank, 'nystrom', threshold)
36             nys_time = time.process_time() - start_time
37
38             # Run RPCholesky PCG with timing
39             start_time = time.process_time()
40             x_rp, _, _ = pcg_solver(A, b, x0, mu, rank, 'rpcholesky', threshold)
41             rp_time = time.process_time() - start_time
42
43             # Store results
44             results['nystrom'][rank]['times'].append(nys_time)
45             results['rpcholesky'][rank]['times'].append(rp_time)
46
47     return results
48
49 def plot_boxplot(results: Dict, ranks: List[int], title: str, output_dir: str = 'plots'):
50     """
51     Create boxplot comparing the CPU time distributions and save as PNG
52     """
53     plt.figure(figsize=(10, 6), dpi=300)
54
55     # Prepare data for boxplot
56     times_data_nystrom = [results['nystrom'][rank]['times'] for rank in ranks]
57     times_data_rpcholesky = [results['rpcholesky'][rank]['times'] for rank in ranks]
58
59     # Position for boxes
60     positions_nystrom = np.arange(len(ranks)) * 2.5
61     positions_rpcholesky = positions_nystrom + 0.8
62
63     # Plot CPU times boxplot
64     plt.boxplot(times_data_nystrom, positions=positions_nystrom, widths=0.6,
65                 patch_artist=True, boxprops=dict(facecolor='lightblue', color='black'),
66                 medianprops=dict(color='red'), labels=[f'l={r}' for r in ranks])
67     plt.boxplot(times_data_rpcholesky, positions=positions_rpcholesky, widths=0.6,
68                 patch_artist=True, boxprops=dict(facecolor='lightgreen', color='black'),
69                 medianprops=dict(color='red'), labels=[f'l={r}' for r in ranks])
70
71     # Customize plot
72     plt.xlabel('Rank')
73     plt.ylabel('CPU Time (seconds)')
74     plt.title(f'CPU Time Distribution — {title}')
75
76     # Add legend
77     from matplotlib.patches import Patch
78     legend_elements = [

```

```

79         Patch(facecolor='lightblue', edgecolor='black', label='Nystr sm'),
80         Patch(facecolor='lightgreen', edgecolor='black', label='RPCholesky')
81     ]
82     plt.legend(handles=legend_elements, loc='upper right')
83
84     plt.tight_layout()
85
86     # Create output directory if it doesn't exist
87     import os
88     os.makedirs(output_dir, exist_ok=True)
89
90     # Save plot as PNG with high DPI
91     filename = os.path.join(output_dir, f'{title.lower().replace(" ", "_")}_boxplot.png')
92     plt.savefig(filename, dpi=300, bbox_inches='tight')
93     plt.close()
94     print(f"Plot saved as {filename}")
95
96 def print_summary_statistics(matrix_name: str, results: Dict, ranks: List[int]):
97     """Print summary statistics for a single experiment"""
98     print(f"\n{matrix_name} Matrix:")
99     for method in ['nystrom', 'rpcholesky']:
100         print(f"\n{method.capitalize()} method:")
101         for rank in ranks:
102             avg_time = np.mean(results[method][rank]['times'])
103             print(f"Rank {rank}: Avg CPU time = {avg_time:.3f} seconds")
104
105 def plot_histogram(results: Dict, ranks: List[int], title: str, output_dir: str = 'plots'):
106     """
107     Create histogram comparing average CPU times between methods and save as PNG
108     """
109     plt.figure(figsize=(10, 6), dpi=300)
110
111     # Calculate averages for each rank
112     nystrom_avgs = [np.mean(results['nystrom'][rank]['times']) for rank in ranks]
113     rpcholesky_avgs = [np.mean(results['rpcholesky'][rank]['times']) for rank in ranks]
114
115     # Set up bar positions
116     x = np.arange(len(ranks))
117     width = 0.35 # Width of the bars
118
119     # Create bars
120     plt.bar(x - width/2, nystrom_avgs, width, label='Nystr sm', color='lightblue', edgecolor='black')
121     plt.bar(x + width/2, rpcholesky_avgs, width, label='RPCholesky', color='lightgreen', edgecolor='black')
122
123     # Customize plot
124     plt.xlabel('Rank')
125     plt.ylabel('Average CPU Time (seconds)')
126     plt.title(f'Average CPU Time Comparison - {title}')
127     plt.xticks(x, [f'l={r}' for r in ranks])
128     plt.legend()
129
130     # Add value labels on top of bars
131     for i, v in enumerate(nystrom_avgs):
132         plt.text(i - width/2, v + 0.01, f'{v:.3f}s', ha='center', va='bottom')
133     for i, v in enumerate(rpcholesky_avgs):
134         plt.text(i + width/2, v + 0.01, f'{v:.3f}s', ha='center', va='bottom')
135
136     plt.tight_layout()
137
138     # Create output directory if it doesn't exist
139     import os
140     os.makedirs(output_dir, exist_ok=True)
141
142     # Save plot as PNG
143     filename = os.path.join(output_dir, f'{title.lower().replace(" ", "_")}_histogram.png')
144     plt.savefig(filename, dpi=300, bbox_inches='tight')
145     plt.close()
146     print(f"Histogram saved as {filename}")
147
148 def run_smile_experiment():
149     """Run experiment for Smile matrix"""
150     # Parameters
151     n = 1000
152     ranks = [50, 75, 100, 125]
153     num_instances = 50
154     threshold = 1e-4
155     mu = 1e-3
156
157     print("Generating Smile matrix...")
158     A_smile = smile(n, bandwidth=2.0)
159
160     print("\nRunning experiments for Smile matrix...")
161     results_smile = run_experiment('Smile', A_smile, ranks, num_instances, threshold, mu)
162
163     print("\nPlotting results...")
164     plot_boxplot(results_smile, ranks, 'Smile Matrix')

```



```

165 plot_histogram(results_smile, ranks, 'Smile Matrix')
166
167 print("\nSummary Statistics:")
168 print_summary_statistics("Smile", results_smile, ranks)
169
170 return results_smile
171
172 def run_spiral_experiment():
173     """Run experiment for Spiral matrix"""
174     # Parameters
175     n = 1000
176     ranks = [50, 75, 100, 125]
177     num_instances = 50
178     threshold = 1e-4
179     mu = 1e-2
180
181     print("Generating Spiral matrix...")
182     A_spiral = robspiral(n)
183
184     print("\nRunning experiments for Spiral matrix...")
185     results_spiral = run_experiment('Spiral', A_spiral, ranks, num_instances, threshold, mu)
186
187     print("\nPlotting results...")
188     plot_boxplot(results_spiral, ranks, 'Spiral Matrix')
189     plot_histogram(results_spiral, ranks, 'Spiral Matrix')
190
191     print("\nSummary Statistics:")
192     print_summary_statistics("Spiral", results_spiral, ranks)
193
194     return results_spiral
195
196 if __name__ == "__main__":
197     import sys
198
199     if len(sys.argv) != 2 or sys.argv[1] not in ['smile', 'spiral', 'both']:
200         print("Usage: python script.py [smile|spiral|both]")
201         sys.exit(1)
202
203     if sys.argv[1] == 'smile' or sys.argv[1] == 'both':
204         results_smile = run_smile_experiment()
205
206     if sys.argv[1] == 'spiral' or sys.argv[1] == 'both':
207         results_spiral = run_spiral_experiment()

```

Listing 8: Third and Fourth Experiment: CPU-time Preconditioners

```

1 import numpy as np
2 from typing import Dict, List, Any
3 import matplotlib.pyplot as plt
4 from PCG import pcg_solver
5 from RPCholesky import rpcholesky
6 from RandomizedNystrom import randomized_nystrom_approximation
7 from HelperFunctions.gallery import smile, robspiral
8 from tqdm import tqdm
9 import time
10
11 def run_experiment(matrix_name: str, A: Any, ranks: List[int],
12                   num_instances: int, threshold: float, mu: float) -> Dict:
13     """
14     Run experiment comparing Nyström PCG with RPCholesky PCG, measuring total CPU time
15     """
16     n = A.shape[0]
17     results = {
18         'nystrom': {r: {'times': []} for r in ranks},
19         'rpcholesky': {r: {'times': []} for r in ranks}
20     }
21
22     # Set random seed for reproducibility
23     np.random.seed(101)
24
25     for rank in ranks:
26         print(f"Processing rank {rank} for {matrix_name}")
27
28         for instance in tqdm(range(num_instances), desc=f"Rank {rank} Instances", position=1, leave=False):
29             # Time Nyström preconditioner construction
30             start_time = time.process_time()
31             U, Lambda = randomized_nystrom_approximation(A, rank)
32             nys_time = time.process_time() - start_time
33
34             # Time RPCholesky preconditioner construction
35             start_time = time.process_time()
36             nystrom_approximation = rpcholesky(A, rank)
37             F = nystrom_approximation.get_left_factor()
38             U, S, _ = np.linalg.svd(F, full_matrices=False)
39             Lambda = S**2

```

```

40         rp_time = time.process_time() - start_time
41
42         # Store results
43         results['nystrom'][rank]['times'].append(nys_time)
44         results['rpcholesky'][rank]['times'].append(rp_time)
45
46     return results
47
48 def plot_boxplot(results: Dict, ranks: List[int], title: str, output_dir: str = 'plots'):
49     """
50     Create boxplot comparing the CPU time distributions and save as PNG
51     """
52     plt.figure(figsize=(10, 6), dpi=300)
53
54     # Prepare data for boxplot
55     times_data_nystrom = [results['nystrom'][rank]['times'] for rank in ranks]
56     times_data_rpcholesky = [results['rpcholesky'][rank]['times'] for rank in ranks]
57
58     # Position for boxes
59     positions_nystrom = np.arange(len(ranks)) * 2.5
60     positions_rpcholesky = positions_nystrom + 0.8
61
62     # Plot CPU times boxplot
63     plt.boxplot(times_data_nystrom, positions=positions_nystrom, widths=0.6,
64                 patch_artist=True, boxprops=dict(facecolor='lightblue', color='black'),
65                 medianprops=dict(color='red'), labels=[f'l={r}' for r in ranks])
66     plt.boxplot(times_data_rpcholesky, positions=positions_rpcholesky, widths=0.6,
67                 patch_artist=True, boxprops=dict(facecolor='lightgreen', color='black'),
68                 medianprops=dict(color='red'), labels=[f'l={r}' for r in ranks])
69
70     # Customize plot
71     plt.xlabel('Rank')
72     plt.ylabel('CPU Time (seconds)')
73     plt.title(f'CPU Time Distribution — {title}')
74
75     # Add legend
76     from matplotlib.patches import Patch
77     legend_elements = [
78         Patch(facecolor='lightblue', edgecolor='black', label='Nyström'),
79         Patch(facecolor='lightgreen', edgecolor='black', label='RPCholesky')
80     ]
81     plt.legend(handles=legend_elements, loc='upper right')
82
83     plt.tight_layout()
84
85     # Create output directory if it doesn't exist
86     import os
87     os.makedirs(output_dir, exist_ok=True)
88
89     # Save plot as PNG with high DPI
90     filename = os.path.join(output_dir, f'{title.lower().replace(" ", "_")}_boxplot.png')
91     plt.savefig(filename, dpi=300, bbox_inches='tight')
92     plt.close()
93     print(f"Plot saved as {filename}")
94
95 def print_summary_statistics(matrix_name: str, results: Dict, ranks: List[int]):
96     """Print summary statistics for a single experiment"""
97     print(f"\n{matrix_name} Matrix:")
98     for method in ['nystrom', 'rpcholesky']:
99         print(f"\n{method.capitalize()} method:")
100         for rank in ranks:
101             avg_time = np.mean(results[method][rank]['times'])
102             print(f"Rank {rank}: Avg CPU time = {avg_time:.3f} seconds")
103
104 def plot_histogram(results: Dict, ranks: List[int], title: str, output_dir: str = 'plots'):
105     """
106     Create histogram comparing average CPU times between methods and save as PNG
107     """
108     plt.figure(figsize=(10, 6), dpi=300)
109
110     # Calculate averages for each rank
111     nystrom_avgs = [np.mean(results['nystrom'][rank]['times']) for rank in ranks]
112     rpcholesky_avgs = [np.mean(results['rpcholesky'][rank]['times']) for rank in ranks]
113
114     # Set up bar positions
115     x = np.arange(len(ranks))
116     width = 0.35 # Width of the bars
117
118     # Create bars
119     plt.bar(x - width/2, nystrom_avgs, width, label='Nyström', color='lightblue', edgecolor='black')
120     plt.bar(x + width/2, rpcholesky_avgs, width, label='RPCholesky', color='lightgreen', edgecolor='black')
121
122     # Customize plot
123     plt.xlabel('Rank')
124     plt.ylabel('Average CPU Time (seconds)')
125     plt.title(f'Average CPU Time Comparison — {title}')

```

```

126 plt.xticks(x, [f'l={r}' for r in ranks])
127 plt.legend()
128
129 # Add value labels on top of bars
130 for i, v in enumerate(nystrom_avgs):
131     plt.text(i - width/2, v + 0.01, f'{v:.3f}s', ha='center', va='bottom')
132 for i, v in enumerate(rpcholesky_avgs):
133     plt.text(i + width/2, v + 0.01, f'{v:.3f}s', ha='center', va='bottom')
134
135 plt.tight_layout()
136
137 # Create output directory if it doesn't exist
138 import os
139 os.makedirs(output_dir, exist_ok=True)
140
141 # Save plot as PNG
142 filename = os.path.join(output_dir, f'{title.lower().replace(" ", "_")}.histogram.png')
143 plt.savefig(filename, dpi=300, bbox_inches='tight')
144 plt.close()
145 print(f"Histogram saved as {filename}")
146
147 def run_smile_experiment():
148     """Run experiment for Smile matrix"""
149     # Parameters
150     n = 1000
151     ranks = [50, 75, 100, 125]
152     num_instances = 50
153     threshold = 1e-4
154     mu = 1e-3
155
156     print("Generating Smile matrix...")
157     A_smile = smile(n, bandwidth=2.0)
158
159     print("\nRunning experiments for Smile matrix...")
160     results_smile = run_experiment('Smile', A_smile, ranks, num_instances, threshold, mu)
161
162     print("\nPlotting results...")
163     plot_boxplot(results_smile, ranks, 'Smile Matrix')
164     plot_histogram(results_smile, ranks, 'Smile Matrix')
165
166     print("\nSummary Statistics:")
167     print_summary_statistics("Smile", results_smile, ranks)
168
169     return results_smile
170
171 def run_spiral_experiment():
172     """Run experiment for Spiral matrix"""
173     # Parameters
174     n = 1000
175     ranks = [50, 75, 100, 125]
176     num_instances = 50
177     threshold = 1e-4
178     mu = 1e-2
179
180     print("Generating Spiral matrix...")
181     A_spiral = robspiral(n)
182
183     print("\nRunning experiments for Spiral matrix...")
184     results_spiral = run_experiment('Spiral', A_spiral, ranks, num_instances, threshold, mu)
185
186     print("\nPlotting results...")
187     plot_boxplot(results_spiral, ranks, 'Spiral Matrix')
188     plot_histogram(results_spiral, ranks, 'Spiral Matrix')
189
190     print("\nSummary Statistics:")
191     print_summary_statistics("Spiral", results_spiral, ranks)
192
193     return results_spiral
194
195 if __name__ == "__main__":
196     import sys
197
198     if len(sys.argv) != 2 or sys.argv[1] not in ['smile', 'spiral', 'both']:
199         print("Usage: python script.py [smile|spiral|both]")
200         sys.exit(1)
201
202     if sys.argv[1] == 'smile' or sys.argv[1] == 'both':
203         results_smile = run_smile_experiment()
204
205     if sys.argv[1] == 'spiral' or sys.argv[1] == 'both':
206         results_spiral = run_spiral_experiment()

```

Listing 9: Conjugate Gradient Method

```

1 import numpy as np
2 from typing import Tuple, List, Any
3
4 def unpreconditioned_cg(A: Any, b: np.ndarray, x0: np.ndarray, mu: float,
5                       eta: float = 1e-6, max_iter: int = 100) -> Tuple[np.ndarray, int, float]:
6     """
7     Implements standard (unpreconditioned) conjugate gradient method for solving regularized linear systems
8
9     Args:
10         A: PSD matrix
11         b: right-hand side vector
12         x0: initial guess
13         mu: regularization parameter
14         eta: solution tolerance
15         max_iter: maximum number of iterations
16
17     Returns:
18         x: Approximate solution to the regularized system
19         num_iters: Number of iterations performed
20         rel_error: Final relative error
21     """
22     # Initialize
23     x = x0
24     r = b - ((A + mu * np.eye(A.shape[0])) @ x)
25     p = r.copy()
26
27     # Main CG loop
28     num_iters = 0
29     for i in range(max_iter):
30         num_iters = i + 1
31
32         Ap = (A + mu * np.eye(A.shape[0])) @ p
33         alpha = (r.T @ r) / (p.T @ Ap)
34         x = x + alpha * p
35         r_new = r - alpha * Ap
36
37         # Check convergence
38         rel_error = np.linalg.norm(r_new) / np.linalg.norm(b)
39         if rel_error <= eta:
40             break
41
42         beta = (r_new.T @ r_new) / (r.T @ r)
43         p = r_new + beta * p
44         r = r_new
45
46     return x, num_iters, rel_error

```

Listing 10: Fifth Experiment: CPU-time Compared to Unpreconditioned System

```

1 import numpy as np
2 from typing import Dict, List, Any
3 import matplotlib.pyplot as plt
4 from PCG import pcg_solver
5 from RPCholesky import rpcholesky
6 from RandomizedNyström import randomized_nystrom_approximation
7 from HelperFunctions.gallery import smile, robspirals
8 from tqdm import tqdm
9 import time
10 import os
11 from CG import unpreconditioned_cg
12
13 def run_experiment(matrix_name: str, A: Any, ranks: List[int],
14                  num_instances: int, threshold: float, mu: float) -> Dict:
15     """
16     Run experiment comparing Nyström PCG, RPCholesky PCG, and unpreconditioned CG,
17     measuring total CPU time
18     """
19     n = A.shape[0]
20     results = {
21         'nystrom': {r: {'times': []} for r in ranks},
22         'rpcholesky': {r: {'times': []} for r in ranks},
23         'unpreconditioned': {r: {'times': []} for r in ranks} # Added for completeness, will use same r
24     }
25
26     # Set random seed for reproducibility
27     np.random.seed(101)
28
29     for rank in ranks:
30         print(f"Processing rank {rank} for {matrix_name}")
31
32         for instance in tqdm(range(num_instances), desc=f"Rank {rank} Instances", position=1, leave=False):
33             # Generate random right-hand side vector
34             b = np.random.randn(n)
35             x0 = np.zeros(n)

```

```

37     # Run Nyström PCG with timing
38     start_time = time.process_time()
39     x_nys, _, _ = pcg_solver(A, b, x0, mu, rank, 'nystrom', threshold)
40     nys_time = time.process_time() - start_time
41
42     # Run RPCholesky PCG with timing
43     start_time = time.process_time()
44     x_rp, _, _ = pcg_solver(A, b, x0, mu, rank, 'rpcholesky', threshold)
45     rp_time = time.process_time() - start_time
46
47     # Run unpreconditioned CG with timing
48     start_time = time.process_time()
49     x_uncond, _, _ = unpreconditioned_cg(A, b, x0, mu, threshold)
50     uncond_time = time.process_time() - start_time
51
52     # Store results
53     results['nystrom'][rank]['times'].append(nys_time)
54     results['rpcholesky'][rank]['times'].append(rp_time)
55     results['unpreconditioned'][rank]['times'].append(uncond_time)
56
57     return results
58
59 def plot_boxplot(results: Dict, ranks: List[int], title: str, output_dir: str = 'plots'):
60     """
61     Create boxplot comparing the CPU time distributions and save as PNG
62     """
63     plt.figure(figsize=(12, 6), dpi=300)
64
65     # Prepare data for boxplot
66     times_data_nystrom = [results['nystrom'][rank]['times'] for rank in ranks]
67     times_data_rpcholesky = [results['rpcholesky'][rank]['times'] for rank in ranks]
68     times_data_uncond = [results['unpreconditioned'][rank]['times'] for rank in ranks]
69
70     # Position for boxes
71     positions_nystrom = np.arange(len(ranks)) * 3.5
72     positions_rpcholesky = positions_nystrom + 0.8
73     positions_uncond = positions_rpcholesky + 0.8
74
75     # Plot CPU times boxplot
76     plt.boxplot(times_data_nystrom, positions=positions_nystrom, widths=0.6,
77                 patch_artist=True, boxprops=dict(facecolor='lightblue', color='black'),
78                 medianprops=dict(color='red'), labels=[f'l={r}' for r in ranks])
79     plt.boxplot(times_data_rpcholesky, positions=positions_rpcholesky, widths=0.6,
80                 patch_artist=True, boxprops=dict(facecolor='lightgreen', color='black'),
81                 medianprops=dict(color='red'), labels=[f'l={r}' for r in ranks])
82     plt.boxplot(times_data_uncond, positions=positions_uncond, widths=0.6,
83                 patch_artist=True, boxprops=dict(facecolor='lightgray', color='black'),
84                 medianprops=dict(color='red'), labels=[f'l={r}' for r in ranks])
85
86     # Customize plot
87     plt.xlabel('Rank')
88     plt.ylabel('CPU Time (seconds)')
89     plt.title(f'CPU Time Distribution — {title}')
90
91     # Add legend
92     from matplotlib.patches import Patch
93     legend_elements = [
94         Patch(facecolor='lightblue', edgecolor='black', label='Nyström'),
95         Patch(facecolor='lightgreen', edgecolor='black', label='RPCholesky'),
96         Patch(facecolor='lightgray', edgecolor='black', label='Unpreconditioned')
97     ]
98     plt.legend(handles=legend_elements, loc='upper right')
99
100     plt.tight_layout()
101     plt.savefig(os.path.join(output_dir, f'{title.lower().replace(" ", "_")}_boxplot.png'),
102                dpi=300, bbox_inches='tight')
103     plt.close()
104
105 def print_summary_statistics(matrix_name: str, results: Dict, ranks: List[int]):
106     """Print summary statistics for a single experiment"""
107     print(f"\n{matrix_name} Matrix:")
108     for method in ['nystrom', 'rpcholesky', 'unpreconditioned']:
109         print(f"\n{method.capitalize()} method:")
110         for rank in ranks:
111             avg_time = np.mean(results[method][rank]['times'])
112             print(f"Rank {rank}: Avg CPU time = {avg_time:.3f} seconds")
113
114 def plot_histogram(results: Dict, ranks: List[int], title: str, output_dir: str = 'plots'):
115     """
116     Create histogram comparing average CPU times between methods and save as PNG
117     """
118     plt.figure(figsize=(12, 6), dpi=300)
119
120     # Calculate averages for each rank
121     nystrom_avgs = [np.mean(results['nystrom'][rank]['times']) for rank in ranks]
122     rpcholesky_avgs = [np.mean(results['rpcholesky'][rank]['times']) for rank in ranks]

```

```

123 uncond_avgs = [np.mean(results['unpreconditioned'][rank]['times']) for rank in ranks]
124
125 # Set up bar positions
126 x = np.arange(len(ranks))
127 width = 0.25 # Width of the bars
128
129 # Create bars
130 plt.bar(x - width, nystrom_avgs, width, label='Nyström', color='lightblue', edgecolor='black')
131 plt.bar(x, rpcholesky_avgs, width, label='RPCholesky', color='lightgreen', edgecolor='black')
132 plt.bar(x + width, uncond_avgs, width, label='Unpreconditioned', color='lightgray', edgecolor='black')
133
134 # Customize plot
135 plt.xlabel('Rank')
136 plt.ylabel('Average CPU Time (seconds)')
137 plt.title(f'Average CPU Time Comparison — {title}')
138 plt.xticks(x, [f'l={r}' for r in ranks])
139 plt.legend()
140
141 # Add value labels on top of bars
142 for i, v in enumerate(nystrom_avgs):
143     plt.text(i - width, v + 0.01, f'{v:.3f}s', ha='center', va='bottom')
144 for i, v in enumerate(rpcholesky_avgs):
145     plt.text(i, v + 0.01, f'{v:.3f}s', ha='center', va='bottom')
146 for i, v in enumerate(uncond_avgs):
147     plt.text(i + width, v + 0.01, f'{v:.3f}s', ha='center', va='bottom')
148
149 plt.tight_layout()
150 plt.savefig(os.path.join(output_dir, f'{title.lower().replace(" ", "_")}.histogram.png'),
151             dpi=300, bbox_inches='tight')
152 plt.close()
153
154 def run_smile_experiment():
155     """Run experiment for Smile matrix"""
156     # Parameters
157     n = 1000
158     ranks = [50, 75, 100, 125]
159     num_instances = 50
160     threshold = 1e-4
161     mu = 1e-3
162
163     print("Generating Smile matrix...")
164     A_smile = smile(n, bandwidth=2.0)
165
166     print("\nRunning experiments for Smile matrix...")
167     results_smile = run_experiment('Smile', A_smile, ranks, num_instances, threshold, mu)
168
169     print("\nPlotting results...")
170     plot_boxplot(results_smile, ranks, 'Smile Matrix')
171     plot_histogram(results_smile, ranks, 'Smile Matrix')
172
173     print("\nSummary Statistics:")
174     print_summary_statistics("Smile", results_smile, ranks)
175
176     return results_smile
177
178 def run_spiral_experiment():
179     """Run experiment for Spiral matrix"""
180     # Parameters
181     n = 1000
182     ranks = [50, 75, 100, 125]
183     num_instances = 50
184     threshold = 1e-4
185     mu = 1e-2
186
187     print("Generating Spiral matrix...")
188     A_spiral = robspiral(n)
189
190     print("\nRunning experiments for Spiral matrix...")
191     results_spiral = run_experiment('Spiral', A_spiral, ranks, num_instances, threshold, mu)
192
193     print("\nPlotting results...")
194     plot_boxplot(results_spiral, ranks, 'Spiral Matrix')
195     plot_histogram(results_spiral, ranks, 'Spiral Matrix')
196
197     print("\nSummary Statistics:")
198     print_summary_statistics("Spiral", results_spiral, ranks)
199
200     return results_spiral
201
202 if __name__ == "__main__":
203     import sys
204
205     if len(sys.argv) != 2 or sys.argv[1] not in ['smile', 'spiral', 'both']:
206         print("Usage: python script.py [smile|spiral|both]")
207         sys.exit(1)
208

```

```

209 if sys.argv[1] == 'smile' or sys.argv[1] == 'both':
210     results_smile = run_smile_experiment()
211
212 if sys.argv[1] == 'spiral' or sys.argv[1] == 'both':
213     results_spiral = run_spiral_experiment()

```

Listing 11: Sixth Experiment: Iteration Count Compared to Unpreconditioned System

```

1 import numpy as np
2 from typing import Dict, List, Any
3 import matplotlib.pyplot as plt
4 from PCG import pcg_solver
5 from RPCholesky import rpcholesky
6 from RandomizedNyström import randomized_nystrom_approximation
7 from HelperFunctions.gallery import smile, robspiral
8 from tqdm import tqdm
9 import os
10 from CG import unpreconditioned_cg
11
12 def run_experiment(matrix_name: str, A: Any, ranks: List[int],
13                     num_instances: int, threshold: float, mu: float) -> Dict:
14     """
15     Run experiment comparing Nyström PCG, RPCholesky PCG, and unpreconditioned CG iterations
16     """
17     n = A.shape[0]
18     results = {
19         'nystrom': {r: {'iters': []} for r in ranks},
20         'rpcholesky': {r: {'iters': []} for r in ranks},
21         'unpreconditioned': {r: {'iters': []} for r in ranks} # Added for consistency with structure
22     }
23
24     for rank in ranks:
25         print(f"\nProcessing rank {rank} for {matrix_name}")
26         # Set random seed for reproducibility
27         np.random.seed(101)
28
29         for instance in tqdm(range(num_instances), desc=f"Rank {rank} Instances", position=1, leave=False):
30             # Generate random right-hand side vector
31             b = np.random.randn(n)
32             x0 = np.zeros(n)
33
34             # Run Nyström PCG
35             x_nys, nys_iters, _ = pcg_solver(A, b, x0, mu, rank, 'nystrom', threshold)
36
37             # Run RPCholesky PCG
38             x_rp, rp_iters, _ = pcg_solver(A, b, x0, mu, rank, 'rpcholesky', threshold)
39
40             # Run unpreconditioned CG
41             x_uncond, uncond_iters, _ = unpreconditioned_cg(A, b, x0, mu, threshold)
42
43             # Store results
44             results['nystrom'][rank]['iters'].append(nys_iters)
45             results['rpcholesky'][rank]['iters'].append(rp_iters)
46             results['unpreconditioned'][rank]['iters'].append(uncond_iters)
47
48     return results
49
50 def plot_boxplot(results: Dict, ranks: List[int], title: str, output_dir: str = 'plots'):
51     """
52     Create boxplot comparing the iteration distributions and save as PNG
53     """
54     plt.figure(figsize=(12, 6), dpi=300)
55
56     # Prepare data for boxplot
57     iters_data_nystrom = [results['nystrom'][rank]['iters'] for rank in ranks]
58     iters_data_rpcholesky = [results['rpcholesky'][rank]['iters'] for rank in ranks]
59     iters_data_uncond = [results['unpreconditioned'][rank]['iters'] for rank in ranks]
60
61     # Position for boxes
62     positions_nystrom = np.arange(len(ranks)) * 3.5
63     positions_rpcholesky = positions_nystrom + 0.8
64     positions_uncond = positions_rpcholesky + 0.8
65
66     # Plot iterations boxplot
67     plt.boxplot(iters_data_nystrom, positions=positions_nystrom, widths=0.6,
68                 patch_artist=True, boxprops=dict(facecolor='lightblue', color='black'),
69                 medianprops=dict(color='red'), labels=[f'l={r}' for r in ranks])
70     plt.boxplot(iters_data_rpcholesky, positions=positions_rpcholesky, widths=0.6,
71                 patch_artist=True, boxprops=dict(facecolor='lightgreen', color='black'),
72                 medianprops=dict(color='red'), labels=[f'l={r}' for r in ranks])
73     plt.boxplot(iters_data_uncond, positions=positions_uncond, widths=0.6,
74                 patch_artist=True, boxprops=dict(facecolor='lightgray', color='black'),
75                 medianprops=dict(color='red'), labels=[f'l={r}' for r in ranks])
76
77     # Customize plot

```

```

78 plt.xlabel('Rank')
79 plt.ylabel('Number of Iterations')
80 plt.title(f'Iterations Distribution — {title}')
81
82 # Add legend
83 from matplotlib.patches import Patch
84 legend_elements = [
85     Patch(facecolor='lightblue', edgecolor='black', label='Nyström'),
86     Patch(facecolor='lightgreen', edgecolor='black', label='RPCholesky'),
87     Patch(facecolor='lightgray', edgecolor='black', label='Unpreconditioned')
88 ]
89 plt.legend(handles=legend_elements, loc='upper right')
90
91 plt.tight_layout()
92 plt.savefig(os.path.join(output_dir, f'{title.lower().replace(" ", "_")}_boxplot.png'),
93             dpi=300, bbox_inches='tight')
94 plt.close()
95
96 def print_summary_statistics(matrix_name: str, results: Dict, ranks: List[int]):
97     """Print summary statistics for a single experiment"""
98     print(f"\n{matrix_name} Matrix:")
99     for method in ['nystrom', 'rpcholesky', 'unpreconditioned']:
100         print(f"\n{method.capitalize()} method:")
101         for rank in ranks:
102             avg_iters = np.mean(results[method][rank]['iters'])
103             print(f"Rank {rank}: Avg iterations = {avg_iters:.1f}")
104
105 def plot_histogram(results: Dict, ranks: List[int], title: str, output_dir: str = 'plots'):
106     """
107     Create histogram comparing average iterations between methods and save as PNG
108     """
109     plt.figure(figsize=(12, 6), dpi=300)
110
111     # Calculate averages for each rank
112     nystrom_avgs = [np.mean(results['nystrom'][rank]['iters']) for rank in ranks]
113     rpcholesky_avgs = [np.mean(results['rpcholesky'][rank]['iters']) for rank in ranks]
114     uncond_avgs = [np.mean(results['unpreconditioned'][rank]['iters']) for rank in ranks]
115
116     # Set up bar positions
117     x = np.arange(len(ranks))
118     width = 0.25 # Width of the bars
119
120     # Create bars
121     plt.bar(x - width, nystrom_avgs, width, label='Nyström', color='lightblue', edgecolor='black')
122     plt.bar(x, rpcholesky_avgs, width, label='RPCholesky', color='lightgreen', edgecolor='black')
123     plt.bar(x + width, uncond_avgs, width, label='Unpreconditioned', color='lightgray', edgecolor='black')
124
125     # Customize plot
126     plt.xlabel('Rank')
127     plt.ylabel('Average Number of Iterations')
128     plt.title(f'Average Iterations Comparison — {title}')
129     plt.xticks(x, [f'l={r}' for r in ranks])
130     plt.legend()
131
132     # Add value labels on top of bars
133     for i, v in enumerate(nystrom_avgs):
134         plt.text(i - width, v + 1, f'{v:.1f}', ha='center', va='bottom')
135     for i, v in enumerate(rpcholesky_avgs):
136         plt.text(i, v + 1, f'{v:.1f}', ha='center', va='bottom')
137     for i, v in enumerate(uncond_avgs):
138         plt.text(i + width, v + 1, f'{v:.1f}', ha='center', va='bottom')
139
140     plt.tight_layout()
141     plt.savefig(os.path.join(output_dir, f'{title.lower().replace(" ", "_")}_histogram.png'),
142                 dpi=300, bbox_inches='tight')
143     plt.close()
144
145
146 def run_smile_experiment():
147     """Run experiment for Smile matrix"""
148     # Parameters
149     n = 1000
150     ranks = [50, 75, 100, 125]
151     num_instances = 50
152     threshold = 1e-4
153     mu = 1e-3
154
155     print("Generating Smile matrix...")
156     A_smile = smile(n, bandwidth=2.0)
157
158     print("\nRunning experiments for Smile matrix...")
159     results_smile = run_experiment('Smile', A_smile, ranks, num_instances, threshold, mu)
160
161     print("\nPlotting results...")
162     plot_boxplot(results_smile, ranks, 'Smile Matrix') # Add boxplot
163     plot_histogram(results_smile, ranks, 'Smile Matrix') # Add histogram

```



```

164
165     print("\nSummary Statistics:")
166     print_summary_statistics("Smile", results_smile, ranks)
167
168     return results_smile
169
170 def run_spiral_experiment():
171     """Run experiment for Spiral matrix"""
172     # Parameters
173     n = 1000
174     ranks = [50, 75, 100, 125]
175     num_instances = 50
176     threshold = 1e-4
177     mu = 1e-2
178
179     print("Generating Spiral matrix...")
180     A_spiral = robspiral(n)
181
182     print("\nRunning experiments for Spiral matrix...")
183     results_spiral = run_experiment('Spiral', A_spiral, ranks, num_instances, threshold, mu)
184
185     print("\nPlotting results...")
186     plot_boxplot(results_spiral, ranks, 'Spiral Matrix')
187     plot_histogram(results_spiral, ranks, 'Spiral Matrix') # Add histogram
188
189     print("\nSummary Statistics:")
190     print_summary_statistics("Spiral", results_spiral, ranks)
191
192     return results_spiral
193
194 if __name__ == "__main__":
195     import sys
196
197     if len(sys.argv) != 2 or sys.argv[1] not in ['smile', 'spiral', 'both']:
198         print("Usage: python script.py [smile|spiral|both]")
199         sys.exit(1)
200
201     if sys.argv[1] == 'smile' or sys.argv[1] == 'both':
202         results_smile = run_smile_experiment()
203
204     if sys.argv[1] == 'spiral' or sys.argv[1] == 'both':
205         results_spiral = run_spiral_experiment()

```